

Thiago Hideki Sato

Desenvolvimento de Sistemas Computacionais Implementação do Sistema Operacional

São José dos Campos - Brasil

Abril de 2018

Thiago Hideki Sato

Desenvolvimento de Sistemas Computacionais

Implementação do Sistema Operacional

Relatório apresentado à Universidade Federal de São Paulo como parte dos requisitos para aprovação na disciplina de Laboratório de Sistemas Computacionais: Sistemas Operacionais.

Docente: Prof. Dr. Tiago de Oliveira

Universidade Federal de São Paulo - UNIFESP

Instituto de Ciência e Tecnologia - Campus São José dos Campos

São José dos Campos - Brasil

Abril de 2018

Resumo

Este trabalho propõe o desenvolvimento de uma das partes do sistema computacional, o sistema operacional (SO). Para isto, será utilizado o processador e o compilador, desenvolvidos nas disciplinas anteriores, com algumas adaptações que foram necessárias. Inicialmente, foi desenvolvido uma BIOS e um *prompt* de comando para que o usuário conseguisse interagir com o sistema. Para realizar tal interação, foi implementado os seguintes gerenciadores: processos, memória, sistemas de arquivos e dispositivos de entrada/saída. Ao decorrer deste relatório, serão apresentados as técnicas e os algoritmos utilizados para o desenvolvimento desse sistema operacional.

Palavras-chave: Sistema operacional. Gerenciador de processo. Gerenciador de memória. Gerenciador de sistemas de arquivos. Gerenciador de dispositivos de entrada/saída. Processador. Compilador.

Lista de ilustrações

Figura 1 – Um processo pode estar nos estados em execução, bloqueado ou pronto. As transições entre esses estados são mostradas	15
Figura 2 – Plataforma de <i>hardware</i> desenvolvida.	19
Figura 3 – Transferência de dados do HD para a memória de dados ou vice-versa.	23
Figura 4 – Tabela de alocação de arquivos.	28
Figura 5 – Organização do HD.	29
Figura 6 – Prompt de comando.	33
Figura 7 – Teste do LCD.	45
Figura 8 – Teste da entrada de dados.	45
Figura 9 – Teste da memória de dados.	45
Figura 10 – Teste do processador.	46
Figura 11 – Identificação do dispositivo de armazenamento principal.	46
Figura 12 – Apresenta o erro na execução da BIOS.	46
Figura 13 – Apresenta que a BIOS foi executada de forma correta.	46
Figura 14 – Resultado da transferência do sistema operacional para a memória de instruções.	47
Figura 15 – Opção para execução de um programa específico.	47
Figura 16 – Resultado do processo.	48
Figura 17 – Opção para execução de um conjunto de programas.	48
Figura 18 – Troca de contexto sendo realizada.	48
Figura 19 – Resultado processo P2.	48
Figura 20 – Resultado processo P3.	49
Figura 21 – Opção para renomear um programa.	49
Figura 22 – Resultado da operação de renomear um programa.	49
Figura 23 – Opção para criar um programa.	49
Figura 24 – Resultado da operação para criação do programa.	50
Figura 25 – Opção para deletar um programa.	50
Figura 26 – Resultado da operação de deletar um programa.	50
Figura 27 – Opção para mostrar programas.	50
Figura 28 – Mensagem de erro: erro.	51
Figura 29 – Mensagem de erro: programa não encontrado.	51
Figura 30 – Mensagem de erro: escolher outro nome.	51

Sumário

1	INTRODUÇÃO	7
2	OBJETIVOS	9
2.1	Geral	9
2.2	Específico	9
3	FUNDAMENTAÇÃO TEÓRICA	11
3.1	Plataforma de <i>hardware</i>	11
3.1.1	Arquitetura monociclo e multiciclo	12
3.2	Compilador	12
3.2.1	Fase de análise	13
3.2.2	Fase de síntese	13
3.3	Sistema Operacional	13
3.3.1	<i>Prompt</i> de comando	14
3.3.2	BIOS	14
3.3.3	Gerenciamento de Processos	14
3.3.4	Escalonamento de CPU	15
3.3.5	Gerenciamento de Memória	16
3.3.6	Gerenciamento do Sistema de Arquivos	17
3.3.7	Gerenciamento de Dispositivos de Entrada/Saída	18
4	DESENVOLVIMENTO	19
4.1	Alterações na plataforma de Hardware	19
4.1.1	Refatoração	19
4.1.2	Memória interna do FPGA	19
4.1.3	Memória de dados e de instruções	20
4.1.4	Entrada de dados	20
4.1.5	Saída de dados	20
4.1.6	Multiprogramação	21
4.1.7	Módulo HD	22
4.1.8	Módulo BIOS	23
4.1.9	Outras alterações	23
4.2	Alterações no compilador	23
4.2.1	Novas funções	23
4.2.2	Banco de registradores	25
4.2.3	Automatização	25

4.3	Sistema Operacional	26
4.3.1	Gerenciamento de processos	26
4.3.2	Gerenciamento de sistema de arquivos	27
4.3.3	Gerenciamento de memória	28
4.3.4	Gerenciamento de dispositivos de entrada/saída	29
4.3.5	Algoritmos para testes	30
4.3.6	Funcionamento da BIOS	30
4.3.7	<i>Prompt</i> de comando	32
4.3.7.1	Executar um programa	36
4.3.7.2	Executar um conjunto de programas	37
4.3.7.3	Renomear um programa	41
4.3.7.4	Criar um programa	41
4.3.7.5	Deletar um programa	43
4.3.7.6	Mostrar programas	43
5	RESULTADOS OBTIDOS E DISCUSSÕES	45
5.1	Funcionamento da BIOS	45
5.2	Funcionamento do prompt de comando	47
5.2.1	Execução de um programa específico	47
5.2.2	Execução de um conjunto de programas	47
5.2.3	Renomear um programa	49
5.2.4	Criar um programa	49
5.2.5	Deletar um programa	50
5.2.6	Mostrar programas	50
5.2.7	Mensagens de erros	51
6	CONSIDERAÇÕES FINAIS	53
	REFERÊNCIAS	55
	APÊNDICES	57
	APÊNDICE A – CÓDIGO FONTE	59
A.1	<i>BIOS.cm</i>	59
A.2	<i>SO.cm</i>	60
	APÊNDICE B – ALGORITMOS PARA TESTES	67
B.1	<i>buscaBinaria.cm</i>	67
B.2	<i>fatorial.cm</i>	67

B.3	<i>fibonacci.cm</i>	68
B.4	<i>mdc.cm</i>	68
B.5	<i>media.cm</i>	68
B.6	<i>mediana.cm</i>	69
B.7	<i>mmc.cm</i>	70
B.8	<i>palindromo.cm</i>	70
B.9	<i>potencia.cm</i>	71
B.10	<i>selectionSort.cm</i>	71

ANEXOS 73

	ANEXO A – CÓDIGO PARA O LCD	75
A.1	lcdlab3.v	75
A.2	LCD_Display.v	76
A.3	reset_delay.v	81
	ANEXO B – DEBOUNCE	83

1 Introdução

Um sistema computacional é formado, basicamente, por *hardware* e *software*. O *hardware* é a parte física do computador, ou seja, os componentes eletrônicos que são responsáveis por executar as instruções. Já o *software* é a parte lógica que processa os dados e executa programas (instruções definidas para resolução do problema).

Para escrever um *software* para determinado *hardware* é necessário conhecer todo o seu conjunto de instruções ou ter um compilador que converte o código em uma linguagem para o código de máquina. Porém, este procedimento atrasa o desenvolvimento, uma vez que é necessário controlar a localização em que o código será guardado no HD (*hard disk*), como e quando que o programa será executado.

O sistema operacional resolve esses problemas. Através de uma interface para o usuário, é possível criar, desenvolver, deletar e utilizar programas, sem se preocupar como o programa será criado, armazenado e executado; realizando a comunicação entre *hardware* e *software* sem muitas preocupações.

Anteriormente, foi desenvolvido um processador (em Laboratório de Sistemas Computacionais: Arquitetura e Organização de Computadores) e um compilador (em Laboratório de Sistemas Computacionais: Compiladores). Para esta disciplina, Laboratório de Sistemas Computacionais: Sistemas Operacionais, será desenvolvido um sistema operacional com gerenciador de processos, de memória, de sistema de arquivos e de dispositivos de entrada/saída; para isto, será utilizado o processador e o compilador implementados em disciplinas anteriores.

2 Objetivos

2.1 Geral

O objetivo geral deste trabalho é desenvolver um sistema operacional (SO) completo com gerenciamento de processos, de memória, de sistemas de arquivos e de dispositivos de entrada/saída. Para a execução deste SO, será utilizada a placa FPGA da Altera; esta placa será programada com o auxílio do *software* Quartus.

Para o desenvolvimento deste sistema operacional, será utilizado o processador e o compilador feitos anteriormente. Sendo assim, será necessário integrar a parte de *hardware* com a parte de *software* para que o SO funcione de forma correta.

2.2 Específico

O desenvolvimento do sistema operacional foi dividido na implementação das seguintes partes do sistema:

- Desenvolver a BIOS.
- Desenvolver o gerenciador de processos;
- Desenvolver o gerenciador de memória;
- Desenvolver o gerenciador de sistemas de arquivos;
- Desenvolver o gerenciador de dispositivos de entrada/saída;
- Desenvolver o *prompt* de comando.

Para a implementação de cada uma destas partes, será utilizado o FPGA para testar e comprovar o funcionamento da parte desenvolvida.

Com relação a BIOS, serão realizados os seguintes testes: verificar e diagnosticar os componentes do sistema; identificar o dispositivo de armazenamento principal; carregar o sistema operacional para a memória principal; ativar o processador para a execução do sistema operacional.

Para testar os gerenciadores (processos, memória, sistemas de arquivos e dispositivos de entrada/saída), será utilizado o *prompt* de comando para verificar a execução das seguintes tarefas: executar um programa específico (sem preempção); executar um conjunto

de programas (multitarefa com preempção) podendo conter até 10 programas; renomear um programa; criar um programa novo; e deletar um programa.

3 Fundamentação Teórica

Nesta seção, será explicado os conceitos utilizados para o desenvolvimento do projeto, o que inclui especificar a plataforma de *hardware*, o compilador e o sistema operacional.

3.1 Plataforma de *hardware*

A plataforma de *hardware* foi implementada na linguagem de descrição de *hardware* verilog e foi compilada no *software* Quartus, gerando um circuito lógico. Este circuito foi programado na placa FPGA (*Field Programmable Gate Array*) DE2-115 Cyclone IV da Altera.

O FPGA possui diversos componentes integrados em sua placa, o que auxilia na criação de diversos projetos com diferentes funções. No caso, a plataforma de *hardware* desenvolvida utiliza um LED, o LCD, os *displays* de 7 segmentos, as chaves (*switches*) e um botão. Todos esses dispositivos auxiliam na comunicação do usuário com o *hardware* implementado.

A plataforma de *hardware* foi baseada na arquitetura MIPS (*Microprocessor Without Interlocked Pipeline Stages*). Assim, o processador é composto basicamente por: contador de programa, unidade lógica e aritmética, banco de registradores, unidade de controle e memória (instruções e dados) ([PATTERSON; HENNESSY, 2005](#)).

O contador de programa (PC) é um registrador de 12 *bits*, responsável por determinar a instrução executada no programa. Este contador é alterado a cada ciclo de clock.

A unidade lógica e aritmética (ULA) é um circuito digital que realiza operações lógicas e aritméticas entre operandos de 32 *bits*. O resultado é enviado para o *datapath* que irá escrever o resultado em um registrador. O *datapath* realiza o caminho dos dados, determinando e enviando os dados para outras unidades do processador.

O banco de registradores armazena 32 registradores de 32 *bits*, estes registradores são responsáveis por armazenar valores para a realização de operações.

A memória de instruções consegue armazenar até 4096 palavras de 32 *bits*. Ela é responsável por guardar instruções que serão executadas no programa, estas instruções contém: o endereço/valor dos operandos e o código referente a instrução (*opcode*).

A memória de dados é responsável por armazenar os dados gerados pela execução do programa. Sua capacidade de armazenamento é a mesma que a da memória de instruções

(4096 palavras de 32 *bits*).

A unidade de controle envia sinais para o restante do processador de acordo com a instrução que está sendo executada. É ela que define a localização dos dados, a escrita na memória ou no banco de registradores, entre outras funções.

Além desses componentes foi emulado um HD (*hard disk*) na plataforma de *hardware*. Teoricamente, o HD é uma memória secundária de acesso mais lento, porém com uma capacidade de armazenamento maior. Porém, como será visto mais a frente, o problema de acesso lento não existirá.

3.1.1 Arquitetura monociclo e multiciclo

Os termos arquitetura monociclo e multiciclo foram citados durante o trabalho, assim, para melhor entendimento será explicado brevemente cada arquitetura.

A arquitetura monociclo executa somente uma instrução a cada ciclo de *clock*. “Os sinais dos *bits* se propagam pelos componentes combinacionais. Neste processo, dados podem ser escritos em elementos sequenciais”. (PASSOS, 2015).

Já na arquitetura multiciclo, a “execução é dividida em etapas (busca da instrução, decodificação, busca de operandos, execução e armazenamento do resultado). Em um ciclo de relógio, apenas uma das etapas é executada para uma dada instrução.” (PASSOS, 2015).

3.2 Compilador

O compilador é:

um programa de sistema que traduz um programa descrito em uma linguagem de alto nível para um programa equivalente em código de máquina para um processador. Em geral, um compilador não produz diretamente o código de máquina mas sim um programa em linguagem simbólica semanticamente equivalente ao programa em linguagem de alto nível. O programa em linguagem simbólica é então traduzido para o programa em linguagem de máquina através de montadores.

Para desempenhar suas tarefas, um compilador deve executar dois tipos de atividade. A primeira atividade é a análise do código fonte, onde a estrutura e significado do programa de alto nível são reconhecidos. A segunda atividade é a síntese do programa equivalente em linguagem simbólica. (RICARTE, 2003).

O compilador implementado foi baseado na linguagem *C*- descrita por Kenneth Loudon (LOUDEN, 2004). As convenções léxicas e a gramática BNF (*Backus-Naur Form*), descritas por ele, foram utilizadas no compilador.

3.2.1 Fase de análise

A fase de análise é dividida em três partes: análise léxica, sintática e semântica.

A análise léxica implementa um autômato finito para realizar o reconhecimento de uma *string* de entrada. Através de uma expressão regular definida, o analisador léxico irá determinar se o *token* é um símbolo válido da linguagem. (RICARTE, 2003). Para auxiliar na análise léxica, foi utilizado a ferramenta *flex* que verifica se as *strings* coincidem com seus respectivos *tokens*.

A análise sintática realiza:

o reconhecimento de sentenças, ou *parsing*, é o procedimento que verifica se uma dada sentença pertence à linguagem gerada por uma gramática livre de contexto. Este procedimento é essencial para um compilador, que deve reconhecer e validar expressões de diversos tipos (declarações, expressões aritméticas, construções de controle de execução) no processo de construção de um código executável equivalente à expressão reconhecida. (RICARTE, 2003).

A ferramenta *Yacc-Bison* foi escolhida para ajudar no processo de análise sintática, reconhecendo padrões e criando a árvore sintática respectiva ao algoritmo escrito em *C*.

Por fim, o analisador semântico irá verificar: os tipos das variáveis; o retorno das funções (verificando se a variável é do mesmo tipo que o retorno da função); a unicidade da declaração de variáveis e de funções.

3.2.2 Fase de síntese

A fase de síntese tem como objetivo gerar o código objeto para o processador-alvo; porém, este processo não ocorre de uma só vez. Primeiro, a árvore obtida pela análise sintática é convertida em um código intermediário que é independente da plataforma de *hardware*. Em seguida, o código intermediário é convertido para o *assembly* desejado. Assim, boa parte do código do compilador pode ser reaproveitado em diversos processadores. (RICARTE, 2003).

3.3 Sistema Operacional

Nesta seção, será explicado alguns conceitos relacionados ao sistema operacional: gerenciamento de processos, gerenciamento de memória, gerenciamento do sistema arquivos e gerenciamento de dispositivos de entrada/saída. Além desses conceitos, será explicado o funcionamento da BIOS e do *prompt* de comando. Todos estes conceitos servirão de base para o entendimento do restante do relatório.

3.3.1 *Prompt* de comando

O *prompt* de comando é:

um programa que emula o campo de entrada de texto de tela de interface de usuário com base com a interface gráfica do usuário (GUI). Ele pode ser usado para executar comandos inseridos e executar funções administrativas avançadas. Ele também pode ser usado para diagnosticar e solucionar alguns tipos de problemas do Windows. (DELL, 2017).

Neste projeto, foi desenvolvido um *prompt* de comando, no qual, o usuário consegue se comunicar com o sistema e realizar alguns comandos.

3.3.2 BIOS

O acrônimo BIOS significa *Basic Input/Output System* (Sistema Básico de Entrada/Saída). O BIOS é um *firmware* incorporado à placa de sistema do seu computador. Quando o computador é iniciado pela primeira vez, o BIOS ativa todo o hardware necessário para o boot do computador, incluindo: *chipset*, processador e cache, memória de sistema (RAM), unidades internas. Após concluir esse processo, o BIOS transfere o controle do computador para o sistema operacional instalado (DELL, 2018).

A BIOS implementada possui algumas dessas características, mas também realiza outras tarefas.

3.3.3 Gerenciamento de Processos

Em sistemas operacionais, o conceito de processo é fundamental.

Um processo é apenas um programa em execução, acompanhado dos valores atuais do contador de programa, dos registradores e das variáveis. Conceitualmente, cada processo tem sua própria CPU virtual. É claro que, na realidade, a CPU troca, a todo momento, de um processo para outro, mas, para entender o sistema, é muito mais fácil pensar em um conjunto de processos executando paralelamente do que tentar controlar o modo como a CPU faz esses chaveamento. Esse mecanismo de trocas rápidas é chamado de *multiprogramação*. (TANENBAUM, 2009, p. 50).

Regularmente, o sistema operacional realiza a alternância de processos, porém, nesta transição é necessário trocar o contexto. Ou seja, salvar os dados atuais para serem utilizados posteriormente quando o processo voltar a ser executado.

Como pode-se observar na Figura 1, o processo pode ter, principalmente, três estados: execução, bloqueado ou pronto. A transição para cada estado é determinado pelo processo em si (solicitação de uma entrada de dados) ou pelo seu escalonador.

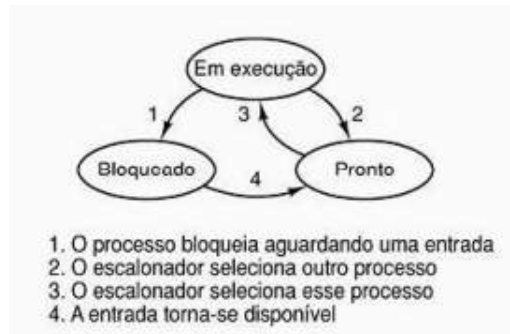


Figura 1 – Um processo pode estar nos estados em execução, bloqueado ou pronto. As transições entre esses estados são mostradas

Fonte: Sistemas Operacionais Modernos ([TANENBAUM, 2009](#))

O escalonador seleciona o próximo processo que está na fila de pronto para ser executado. Para auxiliar nesta tarefa, existem os blocos de controle de processo (*process control block*) que

contém informações sobre o estado do processo, seu contador de programa, o ponteiro da pilha, a alocação de memória, os estados de seus arquivos abertos, sua informação sobre contabilidade e escalonamento e tudo o mais sobre o processo que deva ser salvo quando o processo passar do estado em execução para o estado pronto ou bloqueado, para que ele possa ser reiniciado depois, como se nunca tivesse sido bloqueado. ([TANENBAUM, 2009](#), p. 55).

A seguir será explicado como funciona o escalonador de uma *CPU* (*Central Processing Unit*).

3.3.4 Escalonamento de CPU

Em sistemas multiprogramados,

sempre que dois ou mais processos estão simultaneamente no estado pronto. Se somente uma CPU se encontrar disponível, deverá ser feita uma escolha de qual processo executará em seguida. A parte do sistema operacional que faz a escolha é chamada de **escalonador**, e o algoritmo que ele usa é o **algoritmo de escalonamento** ([TANENBAUM, 2009](#), p. 87).

Os escalonadores podem ser divididos em dois tipos:

- não preemptivos: ao iniciar a execução de um processo, este processo continua sua execução até seja bloqueado ou termine. ([TANENBAUM, 2009](#)).
- preemptivos: inicia a execução de um processo por um período máximo. Ao final desse intervalo, se o processo ainda estiver sendo executado, ele será suspenso e o escalonador escolherá outro processo para ser executado. ([TANENBAUM, 2009](#)).

Existem diversos tipos de escalonadores, segue abaixo alguns destes escalonadores:

- *First come, first served (FCFS)*: primeiro a chegar, primeiro a ser servido. É um escalonador não preemptivo.

A CPU é atribuída aos processos na ordem em que eles a requisitam. (...) À medida que chegam as outras tarefas, eles são encaminhados para o fim da fila. Quando o processo em execução é bloqueado, o primeiro processo na fila é o próximo a executar. Quando um processo bloqueado fica pronto, ele é posto no fim da fila. (TANENBAUM, 2009, p. 92).

- *Shortest job first (SJF)*: tarefa mais curta primeiro. É um escalonador não preemptivo, pois ao executar o processo com menor tempo de execução, a troca de processos só ocorrerá quando o processo liberar voluntariamente a CPU.
- *Shortest remaining time next*: próximo de menor tempo restante. É um escalonador preemptivo.

O escalonador escolhe o processo cujo tempo de execução restante seja o menor. (...) Quando chega uma nova tarefa, seu tempo total é comparado ao tempo restante do processo em curso. Se, para terminar, a nova tarefa precisa de menos tempo que o processo corrente, então esse será suspenso e a nova tarefa será iniciada (TANENBAUM, 2009, p. 93).

- *Round-Robin*: escalonamento por chaveamento circular. É um escalonador preemptivo.

A cada processo é atribuído um intervalo de tempo, o seu *quantum*, no qual ele é permitido executar. Se, ao final do *quantum*, o processo ainda estiver executando, a CPU sofrerá preempção e será dada a outro processo. Se o processo foi bloqueado ou terminou antes que o *quantum* tenha decorrido, a CPU é chaveada para outro processo. (TANENBAUM, 2009, p. 93).

3.3.5 Gerenciamento de Memória

O gerenciador de memória tem como função “gerenciar a memória de modo eficiente: manter o controle de quais partes da memória estão em uso e quais não estão, alocando memória aos processos quando eles precisam e liberando-a quando esses processos terminam.” (TANENBAUM, 2009, p. 106). A alocação de memória é determinada pelo seu algoritmo, sendo que os algoritmos mais conhecidos são:

- *First fit*: primeiro encaixe.

Procura ao longo da lista de segmentos de memória livre que seja suficientemente grande para esse processo. Esse segmento é, então, quebrado em duas partes, uma das quais é alocada ao processo, e a parte restante transforma-se em um segmento de memória livre de tamanho menor (...). (TANENBAUM, 2009, p. 113).

- *Next fit*: próximo encaixa.

Funciona da mesma maneira que o algoritmo *first fit*, exceto pelo fato de sempre memorizar a posição em que encontra um segmento de memória disponível de tamanho suficiente. Quando o algoritmo *next fit* tornar a ser chamado para encontrar um novo segmento de memória livre, ele inicializará sua busca a partir desse ponto, em vez de procurar sempre a partir do início da lista, tal como faz o algoritmo *first fit*. (TANENBAUM, 2009, p. 113).

- *Best fit*: melhor encaixe. “Pesquisa a lista inteira e escolhe o menor segmento de memória livre que seja adequado ao processo” (TANENBAUM, 2009, p. 113).
- *Worst fit*: pior encaixe. “Sempre escolhe o maior segmento de memória disponível, de modo que, quando dividido, o segmento de memória disponível restante, após a alocação ao processo, fosse suficientemente grande para ser útil depois” (TANENBAUM, 2009, p. 113).

3.3.6 Gerenciamento do Sistema de Arquivos

Os arquivos:

são unidades lógicas de informação criadas por processos. Em geral, um disco contém milhares de arquivos, independente do outro. Na verdade, os arquivos também são uma espécie de espaço de endereçamento, mas eles são usados para modelar o disco e não a memória RAM. (TANENBAUM, 2009, p. 158).

Estes arquivos podem ser alocados de diversas formas no HD, dentre elas, temos:

- Alocação contígua: “armazenar cada arquivo em blocos contíguos de disco” (TANENBAUM, 2009, p. 170). Este tipo de alocação é simples de implementar e possui uma leitura rápida. Porém, pode ocorrer fragmentação externa com o tempo.
- Alocação por lista encadeada: armazenar os arquivos “como uma lista encadeada de blocos de discos. A primeira palavra de cada bloco é usada como ponteiro para um próximo. O restante do bloco é usado para dados.” (TANENBAUM, 2009, p. 171). Este tipo de alocação pode utilizar todo o bloco de disco, porém, o acesso aleatório é lento.
- Alocação por lista encadeada usando uma tabela na memória:

todo o bloco fica disponível para dados. Além disso, o acesso aleatório se torna muito mais fácil. Embora ainda seja necessário seguir o encadeamento para encontrar um dado deslocamento dentro do arquivo, o encadeamento permanece inteiramente na memória, de modo que pode ser seguido sem fazer qualquer referência ao disco. Como no método anterior, um inteiro simples (o número do bloco inicial) é o suficiente para representar uma entrada de diretório, permitindo a localização de todos os blocos do arquivo, não importando seu tamanho. (TANENBAUM, 2009, p. 172).

Para a organização destes arquivos, existem os sistemas de arquivos que “são armazenados em discos. A maioria dos discos é dividida em uma ou mais partições, com sistemas de arquivos independentes para cada partição.” (TANENBAUM, 2009, p. 169).

Ao desenvolver o sistema operacional, decidiu-se utilizar o conceito de tabela de alocação de arquivos (FAT, *File Allocation Table*). Esta tabela FAT “é um mapa de utilização do disco. A FAT mapeia a utilização do espaço do disco, ou seja, graças à ela o sistema operacional é capaz de saber onde exatamente no disco um determinado arquivo está armazenado” (TORRES, 1997).

3.3.7 Gerenciamento de Dispositivos de Entrada/Saída

Uma fração substancial de qualquer sistema operacional é relacionada com E/S, que pode ser realizada de três maneiras. Na primeira, existe a E/S programada, na qual a CPU principal lê ou escreve cada byte ou palavra e espera em um laço estreito até que ela possa obter ou enviar o próximo dado. Na segunda, existe uma E/S orientada à interrupção, na qual a CPU inicializa uma transferência de E/S para um caractere ou palavra e segue para outra atividade até que uma interrupção sinalize a conclusão daquela E/S. Na terceira, existe um DMA (*Direct Memory Access*), no qual um chip separado gerencia a transferência completa de um bloco de dados, ocorrendo uma interrupção somente quando o bloco for totalmente transferido. (TANENBAUM, 2009, p. 264).

O gerenciamento de dispositivos de entrada/saída não utilizou nenhum desses princípios citados. A implementação deste gerenciador utilizou o *clock* para controlar a entrada e saída do sistema.

4 Desenvolvimento

Para realizar o desenvolvimento do sistema operacional, foi necessário alterar a plataforma de *hardware* e o compilador. A seguir será apresentado tais alterações e o funcionamento do sistema operacional.

4.1 Alterações na plataforma de Hardware

4.1.1 Refatoração

Primeiramente, foi refatorado o código da plataforma de *hardware*, facilitando o seu entendimento e sua alteração. Ao refatorar, alterou-se a arquitetura de multiciclo para monociclo, além de inserir novos módulos no projeto, como o módulo da BIOS e do HD.

Assim, a nova plataforma de *hardware* é composta por um processador, uma BIOS, uma memória (de instruções e de dados), um HD, um LCD, um LED e *displays* de 7 segmentos. A comunicação entre cada um desses elementos é realizada por canal que realizada a troca de dados. A figura 2 apresenta esta nova plataforma.

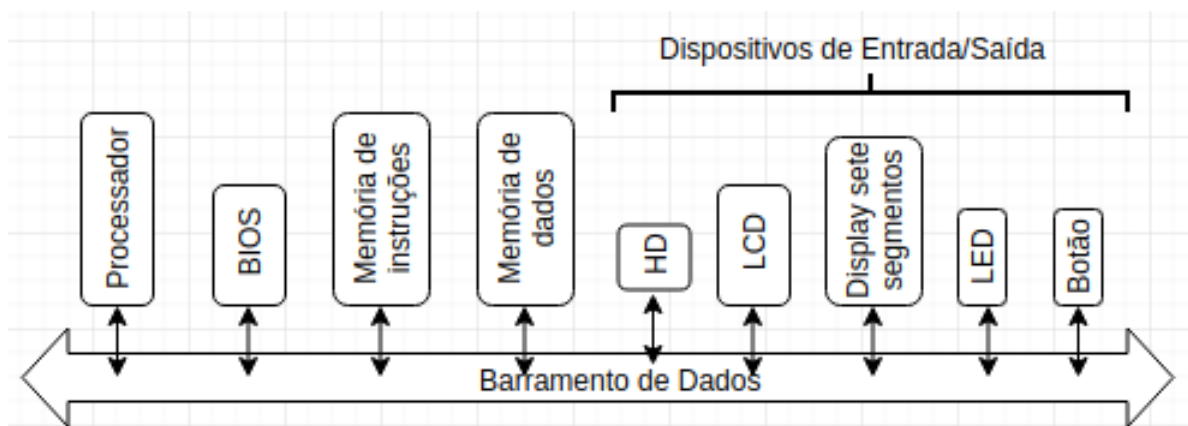


Figura 2 – Plataforma de *hardware* desenvolvida.

4.1.2 Memória interna do FPGA

Um dos problemas encontrados durante o desenvolvimento do sistema operacional foi o tempo de compilação da plataforma de *hardware*. A fim de reduzir este tempo, foram desativados alguns passos da compilação e utilizou-se da memória interna do FPGA.

Para utilizar essa memória interna do FPGA, o próprio *software Quartus* fornece um *template*. Este *template* possui dois *clocks* diferentes: um para leitura e outro para

a escrita. Neste projeto, o *clock* para escrita é o *clock* usado pela própria plataforma de *hardware*, enquanto para leitura foi usado o *clock* interno do FPGA de 50MHz.

A memória interna do FPGA foi implementada no HD, na memória de instruções e na memória de dados.

4.1.3 Memória de dados e de instruções

As memórias de instruções e de dados sofreram uma última modificação. Em cada uma delas, o acesso é determinado por dois valores: o endereço e seu deslocamento. Ao alterar o processo em execução, o deslocamento é alterado. Esta parte será melhor explicada em [gerenciamento de memória](#).

4.1.4 Entrada de dados

Anteriormente, a entrada de dados era realizada somente pelas chaves (*switches*) do FPGA. Agora, esta tarefa é feita pelas chaves e pelo botão. A função deste botão é confirmar o valor de entrada. O módulo *Debounce* ([STOREY, 2013](#)) foi utilizado em sua implementação para melhorar o seu funcionamento. E para facilitar a entrada de dados, o valor obtido pelas chaves é apresentado no *display* de 7 segmentos.

4.1.5 Saída de dados

A saída de dados foi alterada a fim de transmitir melhor a situação do sistema e encontrar possíveis erros na execução dos processos. Para atingir tal objetivo, o código do professor Dr. John S. Loomis referente ao módulo LCD ([LOOMIS, 2008](#)) foi adaptado e utilizado neste projeto.

Neste trabalho, o LCD é capaz de:

- indicar o processo que está em execução. Esta informação precisa ser inserida a partir da instrução *set_num_prog* que define qual processo está sendo executado. Sendo que o processo zero indica o sistema operacional.
- apresentar as saídas dos processos.
- apresentar mensagens. Há valores específicos que apresentam um texto no LCD; estes valores, seus respectivos textos e sua respectiva utilização são apresentados na tabela [1](#).

E para apresentar o valor de saída tempo suficiente para ser observado no LCD, foi implementado a instrução de *delay*. Ela é executada por 750000 ciclos de *clock*, deixando o processador ocioso durante este período. Esta nova instrução é executada logo após uma instrução de saída.

Tabela 1 – Mensagens apresentadas no LCD de acordo com o valor passado.

Valor	Texto	Utilização
65535	OK	BIOS e Sistema Operacional
65534	ERROR	BIOS e Sistema Operacional
65533	OPTION	Sistema Operacional
65532	CS	Sistema Operacional
65531	EXEC	Sistema Operacional
65530	EXEC_N	Sistema Operacional
65529	RENAME	Sistema Operacional
65528	CREATE	Sistema Operacional
65527	DELETE	Sistema Operacional
65526	SHOW	Sistema Operacional
65525	OtherNam	Sistema Operacional
65524	NotFound	Sistema Operacional

4.1.6 Multiprogramação

O processador desenvolvido anteriormente é capaz de executar somente uma instrução por vez; conseqüentemente, não é possível executar dois ou mais processos concorrentemente. Para contornar este problema, foi utilizado o conceito de multiprogramação. Desta forma, não é necessário que um processo termine para começar outro; assim, a execução dos processos ocorre de forma alternada.

Para realizar a multiprogramação, foi preciso criar as seguintes instruções na plataforma de *hardware*:

- *set_multiprog*: define o novo valor do registrador *multProg* que é responsável pela multiprogramação. Caso o valor deste registrador seja zero, a multiprogramação está desativada; caso seja um, a multiprogramação está ativada.
- *set_quantum*: define o valor do registrador *quantum*. Este registrador define a frequência que a troca de processos deve ocorrer.
- *set_addr_cs*: define o endereço que realizará a troca de contexto dos processos.
- *exec_process*: inicia ou continua a execução de um processo. Esta instrução contém a posição de três registradores: o primeiro registrador armazena o valor do novo contador de programa (PC), o segundo guarda o deslocamento da memória de instruções e o terceiro possui o deslocamento da memória de dados.
- *get_pc_process*: obtém o contador de programa do último processo que foi preemptado e armazena o valor no banco de registradores.

Além de criar estas instruções, foi implementado um contador no *datapath* para indicar quando deve ocorrer a troca de contexto. Este contador será melhor explicado em [gerenciamento de processos](#).

4.1.7 Módulo HD

O módulo HD foi implementado com o objetivo de emular o funcionamento de um HD real. Ou seja, uma memória secundária e não volátil que é capaz de armazenar uma grande quantia de dados. No caso, o módulo HD utilizou-se da memória interna do FPGA; por isso, o tempo para acessá-lo será menor que o tempo para executar uma instrução.

O HD implementado suporta até 16384 palavras de 32 *bits* e armazena as seguintes informações: instruções do sistema operacional e dos programas, tabela de alocação de arquivos e alguns dados.

No módulo desenvolvido, o acesso ao HD é realizado pelo setor e pela trilha. Sendo que há 16 setores e para cada setor há 1024 trilhas. Consequentemente, o HD pode armazenar até 16384 dados de 32 *bits*.

Ao inserir este novo módulo, foi necessário adicionar novas instruções que permitissem que o HD se comunicasse com o restante da plataforma de *hardware*. Assim, as seguintes instruções foram criadas:

- *hd_transfer_mi*: transfere o dado lido do HD obtido pelo setor e pela trilha e salva na memória de instruções.
- *save_rf_hd*: salva o valor de um registrador no HD.
- *save_rf_hd_ind*: salva o valor de um registrador no HD. A posição deste registrador é obtido pelo valor de outro registrador.
- *rec_rf_hd*: obtém o dado do HD e o salva em um registrador.
- *rec_rf_hd_ind*: obtém o dado do HD e salva em um registrador. A posição deste registrador é obtido pelo valor de outro registrador.

Todas estas instruções de acesso ao HD utilizam o banco de registradores para obter o valor do setor e da trilha. Assim, ao executar qualquer uma dessas instruções, é preciso carregar estes valores nos registradores para, posteriormente, realizar as operações com o HD.

Analisando essas novas instruções, percebe-se que o HD se comunica somente com o banco de registradores e com a memória de instruções. E caso seja necessário transferir um dado do HD para a memória de dados, é necessário que este dado passe primeiro pelo banco de registradores. A figura 3 explica melhor tal observação.

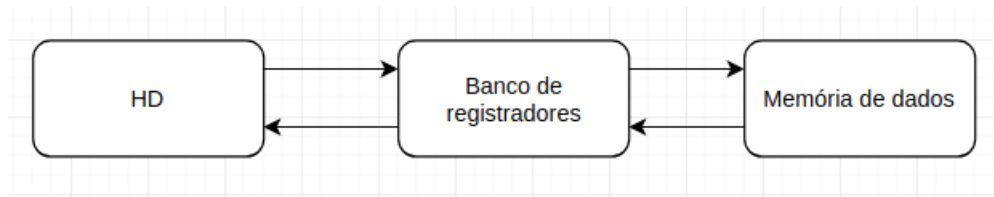


Figura 3 – Transferência de dados do HD para a memória de dados ou vice-versa.

4.1.8 Módulo BIOS

O módulo da BIOS foi criado para armazenar o seu código que foi criado em *cimus*. O programa da BIOS será apresentado em [funcionamento da BIOS](#).

4.1.9 Outras alterações

1. As instruções de multiplicação e de divisão da linguagem *verilog* foram removidas, pois ao compilar com essas operações na plataforma de *hardware*, o *software Quartus* travada e não terminava a sua compilação.
2. Para terminar a execução da instrução *halt*, é preciso que o botão seja pressionado. Essa alteração foi realizada a fim de analisar o funcionamento da multiprogramação; desta forma, quando um processo termina sua execução, é possível verificar o valor final de saída.
3. Com a inserção do módulo da BIOS, a instrução pode vir da memória de instruções ou da BIOS. Assim, foi criado um multiplexador para determinar de onde vem a instrução.

4.2 Alterações no compilador

Além de alterar a plataforma de *hardware*, o compilador também precisou ser modificado: novas funções foram adicionados; o uso do banco de registradores é dividido de forma explícita entre o processo do sistema operacional e outros processos; e a geração do código objeto é realizada de forma automatizada.

4.2.1 Novas funções

1. *HD_Transfer_MI*: transfere o dado lido do HD dado pelo setor e pela trilha e salva na memória de instruções.

Parâmetros da função: endereço da memória de instruções; setor e trilha do HD.

Uso: *HD_Transfer_MI(endMI, setor, trilha)*;

2. *HD_Write*: armazena um valor no HD.
Parâmetros da função: valor a ser guardado no HD; setor e trilha do HD.
Uso: *HD_Write(valor, setor, trilha)*;
3. *HD_Read*: esta função retorna o valor lido do HD.
Parâmetros da função: setor e trilha do HD.
Uso: *a = HD_Read(setor, trilha)*;
4. *saveRF_in_HD*: armazena o valor do registrador no HD.
Parâmetros da função: endereço do banco de registradores; setor e trilha do HD.
Uso: *saveRF_in_HD(endBancoReg, setor, trilha)*;
5. *recoveryRF*: armazena o valor lido do HD em um registrador.
Parâmetros da função: endereço do banco de registradores; setor e trilha do HD.
Uso: *recoveryRF(endBancoReg, setor, trilha)*;
6. *setMultiprog*: define se vai ocorrer ou não a multiprogramação.
Parâmetro da função: valor para indicar se ocorrerá ou não a multiprogramação. Se o valor for igual a zero então a multiprogramação é desativada; caso o valor seja igual a um, a multiprogramação é ativada.
Uso: *setMultiprog(valor)*;
7. *setQuantum*: define o valor do *quantum*.
Parâmetro da função: o novo valor do *quantum*.
Uso: *setQuantum(quantum)*;
8. *setAddrCS*: define o valor do endereço de troca de contexto.
Parâmetros da função: novo endereço que realizará a troca de contexto.
Uso: *setAddrCS(endereço)*;
9. *setNumProg*: define o nome do programa que está sendo executado. Este nome é apresentado no LCD.
Parâmetro da função: nome do programa que está sendo executado.
Uso: *setNumProg(nomePrograma)*;
10. *execProcess*: executa o processo que está na memória de instruções.
Parâmetro da função: novo valor do contador de programa (*novoPC*); valor de deslocamento da memória de instruções *shiftMI*; e valor de deslocamento da memória de dados *shiftMD*.
Uso: *execProcess(novoPC, shiftMI, shiftMD)*;
11. *getPC_Process*: retorna o endereço do PC (contador de programa) do último processo que sofreu preempção.

Parâmetro da função: não há parâmetros.

Uso: `a = getPC_Process();`

12. `returnMain()`: retorna para o início da execução do *prompt* de comando.

Parâmetro da função: não há parâmetros.

Uso: `returnMain();`

4.2.2 Banco de registradores

O processador desenvolvido possui 32 registradores para o banco de registradores, porém, sem diferenciação entre cada um deles. Ao desenvolver o compilador, definiu-se que o banco de registradores ficasse organizado como apresentado na tabela 2.

Tabela 2 – Organização do banco de registradores.

Número	Tipo	Função
0	Zero	Guarda a constante zero
1-5	-	-
6	Entrada/Saída	Utilizado para guardar os valores de entrada/saída
7	Valor de retorno	Guarda o valor de retorno da função
8-13	T	Guarda os valores temporários do processo
14-19	S	Guarda os valores das variáveis do processo
20-25	T	Guarda os valores temporários do sistema operacional
26-31	S	Guarda os valores das variáveis do sistema operacional

Com esta organização do banco de registradores, o sistema operacional possui um espaço reservado para sua execução. Assim, quando a função de troca de contexto é executada, o sistema operacional não irá alterar o valor dos registradores de outros processos.

4.2.3 Automação

O código do compilador foi alterado para automatizar a compilação do código do sistema operacional. Ao executar o compilador, este já lê todos os programas (BIOS, sistema operacional e algoritmos) e já gera os arquivos referentes a BIOS e ao HD. Além disso, o próprio compilador inicia o sistema de arquivos, criando a tabela de alocação de arquivos no HD.

O funcionamento do sistema de arquivos será esclarecido em [gerenciamento de sistema de arquivos](#).

4.3 Sistema Operacional

Para a implementação do sistema operacional (SO), foi utilizado tanto a plataforma de *hardware* quanto o compilador. A seguir será descrito os gerenciadores (processo, memória, arquivos e entrada/saída) desenvolvidos no sistema operacional. Posteriormente, será apresentado os algoritmos utilizados para testes e, por fim, será explicado o funcionamento da BIOS e do *prompt* de comando.

4.3.1 Gerenciamento de processos

Para realizar o gerenciamento de processos, foi implementado o escalonador *Round-Robin*. Durante a implementação deste escalonador, houve a necessidade de criar novas instruções. Estas novas instruções auxiliaram: na definição da multiprogramação; na definição do valor do *quantum* (utilizado no escalonador); na definição do endereço para tratar a troca de contexto; na execução dos processos; e na obtenção do contador de programa do processo preemptado.

No escalonador implementado, a preempção é determinada pelo valor do *quantum*. No processador, há um contador (denominado *contQ*) no *datapath* que é incrementado a cada subida de *clock* quando a multiprogramação está ativada. Quando este contador atinge o valor do *quantum*, o processo para sua execução e inicia a troca de contexto.

A determinação do *quantum* é definida pelo SO desenvolvido. No momento, o *quantum* está fixado em 50 instruções. Porém, para iniciar a execução de um processo, é necessário em torno de 20 instruções até executar efetivamente o processo. Desta forma, apesar do *quantum* ser 50, o processo é executado por 30 ciclos de *clock* até ser preemptado.

A troca de contexto é uma função do sistema operacional implementada em *C*- que realiza as seguintes tarefas:

- Verifica se o processo ainda precisa ser executado;
- Salva o banco de registradores no HD. A posição inicial no HD para salvar estes valores é dada pela última instrução do programa;
- Apresenta uma mensagem indicando que está executando a troca de contexto;
- Busca um novo processo para ser executado. Caso haja algum processo para ser executado, recupera o estado anterior do banco de registradores e apresenta o seu nome no LCD. Caso contrário, volta para a *main*.

Como pode ser observado, a função de troca de contexto necessita de algumas informações dos processos. Assim, há um vetor no sistema operacional que possui as seguintes informações de cada processo:

- Contador do programa atual do processo: para saber de onde continuar sua execução;
- Nome do programa: para apresentar o seu nome no LCD;
- Se está em execução: para saber se é preciso ou não executar o processo;
- Posição base na memória de instruções e de dados: para não acessar a posição de memória errada. A explicação desta parte será melhor abordada em [gerenciamento de memória](#);
- Quantidade de instruções do programa: auxilia na determinação do processo ter terminado ou não;
- Setor e trilha (posição dada pela última instrução do programa): utilizado para salvar ou para recuperar o banco de registradores;
- Se é a primeira vez a ser executado: evita que o LCD apresenta valores de saída de outros processos, iniciando a saída com o valor zero.

4.3.2 Gerenciamento de sistema de arquivos

O gerenciamento do sistema de arquivos é realizado por uma tabela de alocação de arquivos que ocupa as primeiras 1024 posições do HD (figura 4). O início desta tabela apresenta informações gerais, como quantidade de programas existentes no HD, o tamanho máximo dos programas e a próxima posição para escrita no HD. Em seguida, é apresentado as informações do programa: seu uso (indica se há algum programa neste bloco do HD), seu nome, sua posição inicial no HD, sua quantidade de instruções e seu espaço utilizado para a memória de dados.

Como o compilador conhece todas as informações dos programas, decidiu-se automatizar a geração da tabela de alocação de arquivos. Inicialmente, o próprio compilador insere as informações da tabela de alocação de arquivos. Posteriormente, o código do sistema operacional realiza as atualizações.

Para renomear um arquivo, basta procurá-lo e alterar o seu campo de nome. Para deletar um arquivo, é preciso alterar o valor da informação de uso daquele bloco do HD. E para criar, adiciona-se as informações do programa, como nome e início, no primeiro bloco livre da tabela de alocação de arquivos; além de atualizar as informações gerais desta tabela: quantidade de programas e próxima posição para escrita no HD.

Com relação a organização do espaço livre do HD, há um desperdício (ver figura 5). O sistema operacional reserva 2048 posições do HD e só utiliza 1270 posições. Já os programas reservam 1024 posições e a quantidade utilizada depende do programa, mas todos eles utilizam menos do que é reservado. Ou seja, cada arquivo é alocado em uma partição de tamanho fixo.

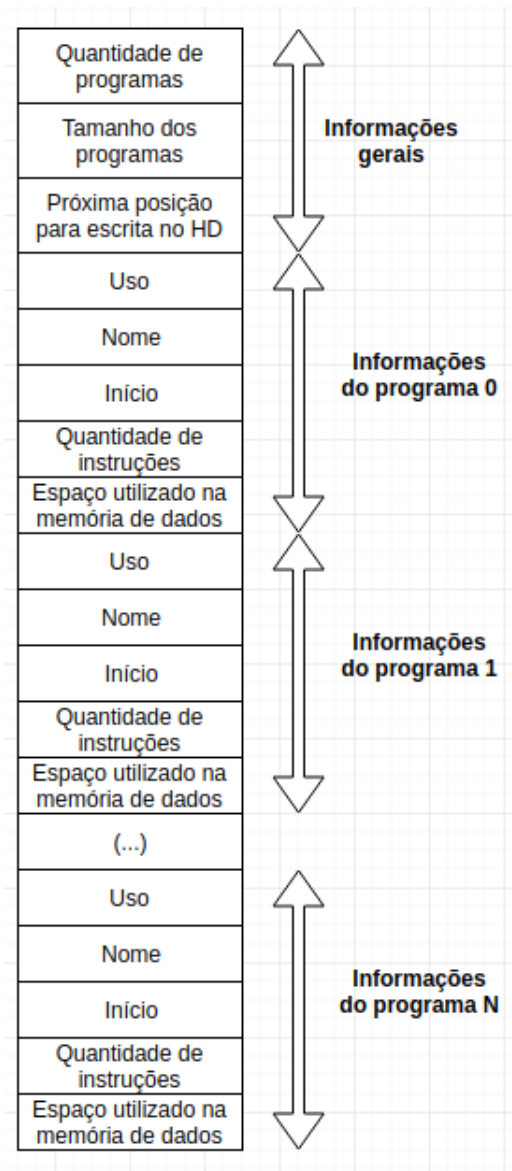


Figura 4 – Tabela de alocação de arquivos.

4.3.3 Gerenciamento de memória

O gerenciador de memória é utilizado da mesma forma para a memória de instruções e para a memória de dados. Assim, ao explicar o gerenciamento de uma dessas memórias; a outra pode ser entendida da mesma forma.

O algoritmo *first fit* foi usado para determinar a região em que as instruções seriam transferidas do HD para a memória de instruções; ou seja, a primeira posição disponível na memória de instruções definiria o endereço base do processo.

Ao executar a BIOS e o sistema operacional, o endereço base será zero. Porém, ao executar um ou mais processos, este deslocamento será definido pelo SO. Sabendo quais os processos que estão sendo executados, é simples obter este deslocamento.

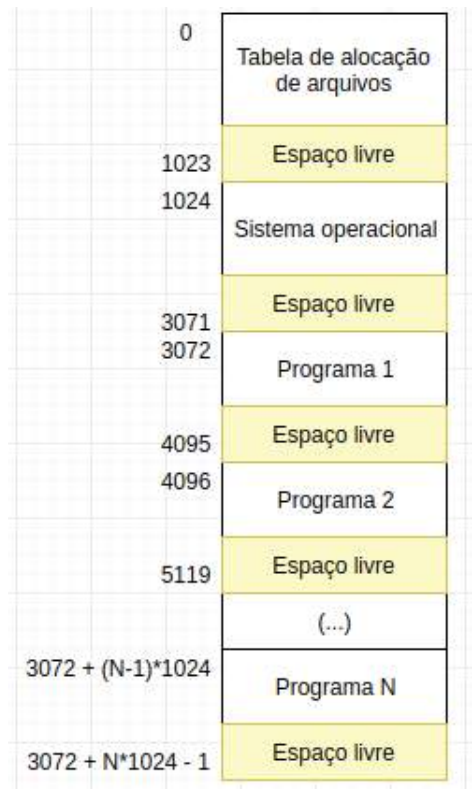


Figura 5 – Organização do HD.

Caso a memória de instruções esteja executando somente o sistema operacional e um novo processo é carregado, o endereço base deste processo será o espaço de memória utilizado pelo SO. Já para a execução de dois processos, o primeiro processo teria o endereço base determinado pelo espaço de memória utilizado pelo SO; porém, o segundo teria um deslocamento igual a soma do espaço de memória do SO com o espaço de memória do primeiro processo. Ao terminar a execução destes processos, o próximo endereço base que pode ser utilizado voltaria a ser determinado pelo espaço de memória do sistema operacional.

O endereço base é de suma importância para a multiprogramação. Sem estes valores, o acesso à memória ocorreria de forma equivocada. A execução de um novo processo é dada pela função *execProcess* do compilador; nesta função, é passado o contador de programa, o deslocamento da memória de instruções e de dados. É por esta razão que o vetor de troca de contexto armazena tais informações.

4.3.4 Gerenciamento de dispositivos de entrada/saída

O desenvolvimento do gerenciador de dispositivos de entrada/saída seguiu uma abordagem diferente do usual e será explicado abaixo.

Para o gerenciamento de entrada, será utilizado o *clock* para controlar o processador. Desta forma, a instrução de entrada irá suspender a execução de instruções até que esta seja

completada; esta instrução só é finalizada quando o botão de confirmação for pressionado.

A instrução de entrada utiliza alguns dispositivos para demonstrar seu funcionamento: o LED, o *display* de sete segmentos e as chaves (*switches*). O *display* de sete segmentos irá mostrar o valor obtido pelas chaves (*switches*). Já o LED irá acender quando for necessário entrar com um valor para a instrução.

Já para a saída, será utilizado o LCD. O LCD irá apresentar o resultado da execução do sistema computacional: indicando qual programa que está sendo executado e seu estado de execução (apresentando a saída de dados do processo). Como discutido anteriormente, a tabela 1 apresenta os textos que podem ser mostradas no LCD.

Por último, o HD será emulado, sendo implementado em *verilog*. Neste HD, serão armazenados os programas e os dados. E com relação a sua leitura/escrita de dados, a localização será indicada pela trilha e pelo setor.

4.3.5 Algoritmos para testes

Para verificar o funcionamento do sistema operacional, foi implementado os seguintes algoritmos para testes: busca binária (B.1), fatorial (B.2), fibonacci (B.3), máximo divisor comum (B.4), média (B.5), mediana (B.6), mínimo múltiplo comum (B.7), palíndromo (B.8), potência (B.9) e ordenação (B.10).

4.3.6 Funcionamento da BIOS

A BIOS é responsável por realizar os testes dos componentes, por carregar o sistema operacional na memória de instruções e por iniciar a execução do sistema operacional. Nesta seção, será explicado o funcionamento da BIOS parte por parte. O seu código completo pode ser encontrado em A.1.

```
1  int ERRO;  
2  int OK_LCD;  
3  int ERROR_LCD;  
4  
5  void main(void){  
6      setMultiprog(0);  
7      delay();  
8      OK_LCD = 65535;  
9      ERROR_LCD = 65534;  
10     ERRO = 1;  
11     while(ERRO == 1) LED();  
12     ERRO = 1;  
13     while(ERRO == 1) entrada();  
14     ERRO = 1;  
15     while(ERRO == 1) memoriaDeDados();  
16     ERRO = 1;  
17     while(ERRO == 1) processador();  
18     ERRO = 1;  
19     while(ERRO == 1) hd_test();  
20     hd_instr();
```



```
21 output(OK_LCD);  
22 }
```

A função *main* realiza as seguintes tarefas:

1. Garante que a BIOS não seja executada com multiprogramação, definindo seu valor como zero;
2. Facilita a impressão de mensagens: as variáveis *OK_LCD* e *ERROR_LCD* são iniciadas de acordo com a tabela 1;
3. Testa o LCD;
4. Testa a entrada de dados;
5. Testa a memória de dados;
6. Testa o processador;
7. Verifica o HD;
8. Carrega o sistema operacional na memória de instruções;
9. Inicia a execução do sistema operacional.

```
1 void confirmacao(){  
2     int conf;  
3     conf = input();  
4     if(conf == 1){  
5         output(OK_LCD);  
6         ERRO = 0;  
7     }  
8     else output(ERROR_LCD);  
9 }
```

A função *confirmacao()* é utilizada em testes e tem como objetivo verificar se o componente está funcionando corretamente. Caso o valor apresentado seja igual ao esperado, deve-se entrar com valor 1. Assim, o teste é concluído e a execução da BIOS pode ser continuada.

```
1 void LED(){  
2     output(3);  
3     confirmacao();  
4 }
```

A função *LED()* testa o funcionamento do LED, apresentando o valor 3 nele.

```
1 void entrada(){  
2     int n;  
3     n = input();  
4     output(n);  
5     confirmacao();  
6 }
```

A função *entrada()* testa a entrada de dados. Primeiro, o valor de entrada é armazenado na variável *n* para depois ser apresentado no LCD.

```
1 void memoriaDeDados(){
2     int n;
3     n = 4;
4     output(n);
5     confirmacao();
6 }
```

A função *memoriaDeDados()* testa a memória de dados. O valor 4 é armazenado na variável *n* (memória de dados) para depois ser apresentado no LCD.

```
1 void processador(){
2     output(3+5);
3     confirmacao();
4 }
```

A função *processador()* testa o processador, verificando o seu funcionamento com uma operação de soma e apresentado o resultado no LCD.

```
1 void hd_test(){
2     HD_Write(10, 15, 1023);
3     output(HD_Read(15, 1023));
4     confirmacao();
5 }
```

A função *hd_test()* verifica o HD. É escrito o valor 10 no HD, depois este valor é lido a partir do HD e apresentado no LCD.

```
1 void hd_instr(){
2     int i; int j;
3     j = 0;
4     for(i = 0; i < 2048; i = i+1){
5         if(i < 1024)
6             HD_Transfer_MI(i, 1, i);
7         else{
8             HD_Transfer_MI(i, 2, j);
9             j = j + 1;
10        }
11    }
12 }
```

A função *hd_instr()* carrega o sistema operacional na memória de instruções. O sistema operacional está armazenado no segundo e no terceiro setor do HD; assim, a função *HD_Transfer_MI* transfere as instruções destes setores para a memória de instruções.

Ao terminar a execução da BIOS, o contador de programa é zerado e a instrução passa a vir do módulo da memória de instruções. Desse modo, o sistema operacional começa a ser executado.

4.3.7 *Prompt* de comando

Nesta seção, será apresentado o funcionamento do *prompt* de comando. Para iniciar a explicação, a figura 6 apresenta como o *prompt* funciona de forma simplificada. O código

completo do *prompt* de comando pode ser encontrado em [A.2](#).

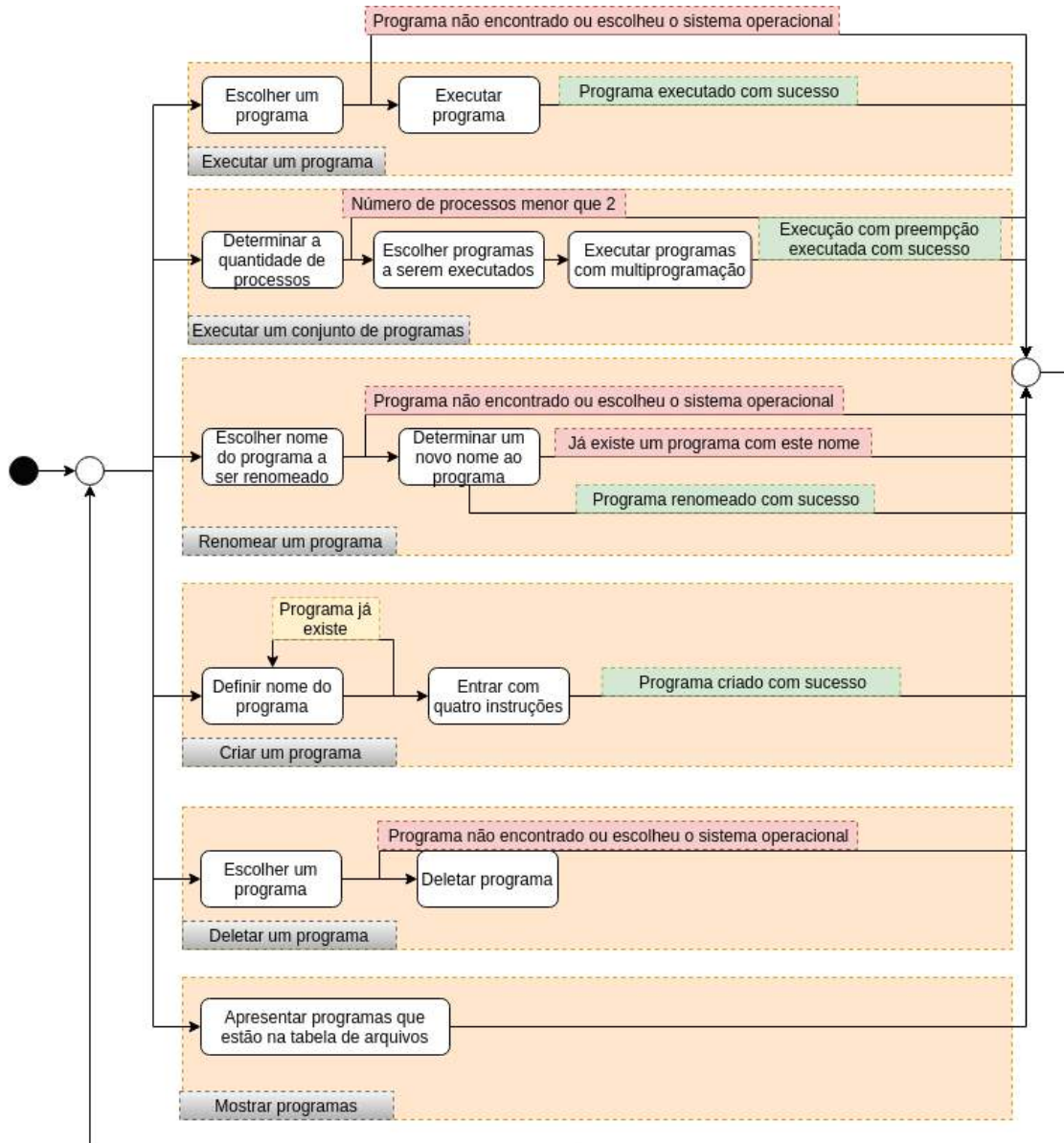


Figura 6 – Prompt de comando.

```

1  int OK_LCD;
2  int ERROR_LCD;
3  int OPTION_LCD;
4  int CS_LCD;
5  int EXECUTE_LCD;
6  int EXECUTE_N_LCD;
7  int RENAME_LCD;
8  int CREATE_LCD;
9  int DELETE_LCD;
10 int SHOW_LCD;
11 int OTHER_NAME_LCD;
12 int NOT_FOUND_LCD;
13 int tab_inicio; int tab_space_program;
14 int exec; int space_to_process; int qtd_processos_ativos;
15 int tam_vector_CS; int n_process_exec; int CS[100];
16 int startRF; int endRF;

```

```

17
18 void main(){
19     int option; int program;
20     setMultiprog(0);
21     OK_LCD = 65535; ERROR_LCD = 65534; OPTION_LCD = 65533; CS_LCD = 65532;
22     EXECUTE_LCD = 65531; EXECUTE_N_LCD = 65530; RENAME_LCD = 65529;
23     CREATE_LCD = 65528; DELETE_LCD = 65527; SHOW_LCD = 65526;
24     OTHER_NAME_LCD = 65525; NOT_FOUND_LCD = 65524;
25     startRF = 6; endRF = 19;
26     setNumProg(0);
27     setAddrCS(1);
28     setQuantum(50);
29     tab_inicio = 3; tab_space_program = 5;
30     space_to_process = 9;
31     while (1 == 1){
32         output(OPTION_LCD);
33         option = input();
34         if(option == 1){
35             output(EXECUTE_LCD);
36             program = input();
37             executeProgram(program);
38         }
39         else if(option == 2){
40             output(EXECUTE_N_LCD);
41             executeNPrograms();
42         }
43         else if(option == 3){
44             output(RENAME_LCD);
45             program = input();
46             output(program);
47             output(renameProgram(program));
48         }
49         else if(option == 4){
50             output(CREATE_LCD);
51             output(createProgram());
52         }
53         else if(option == 5){
54             output(DELETE_LCD);
55             program = input();
56             output(deleteProgram(program));
57         }
58         else if(option == 6){
59             output(SHOW_LCD);
60             showPrograms();
61         }
62     }
63 }

```

A *main* do *prompt* de comando realiza as seguintes tarefas:

1. Garante que a BIOS não seja executada com multiprogramação, definindo seu valor como zero;
2. Facilita a impressão de mensagens: as variáveis *OK_LCD*; *ERROR_LCD*, *OPTION_LCD*, *CS_LCD*, *EXECUTE_LCD*, *EXECUTE_N_LCD*, *RENAME_LCD*, *CREATE_LCD*, *DELETE_LCD*, *SHOW_LCD*, *OTHER_NAME_LCD* e *NOT*

`_FOUND_LCD` são inicializadas de acordo com a tabela 1;

3. Estabele o intervalo do banco de registradores que deve ser salvo no HD ao ocorrer a troca de contexto ($startRF = 6$ e $endRF = 19$);
4. Define que o sistema operacional está sendo executado;
5. Define o endereço para troca de contexto como 1;
6. Define o valor do *quantum* como 50;
7. Estabelece o início do primeiro programa da tabela de alocação de arquivos ($tab_inicio = 3$), além de fixar o espaço ocupado por um programa nesta tabela ($tab_space_program = 6$);
8. Estabelece o espaço ocupado por um processo no vetor *CS* que controla a troca de contexto ($space_to_process = 9$);
9. Imprime a mensagem *OPTION* para indicar que pode ser escolhido uma tarefa a ser executada: 1) executar um programa; 2) executar um conjunto de programas; 3) renomear um programa; 4) criar um programa; 5) deletar um programa; 6) mostrar os programas que estão no HD. Caso o valor de entrada seja diferente desses valores, o usuário volta para a escolha da tarefa.

A seguir, as funções *findProgram* e *loadProgram* serão explicadas. Estas duas funções são utilizadas em diversas partes do código; portanto, entendê-las é essencial para a compreensão do funcionamento do *prompt* de comando.

```

1 int findProgram(int program){
2     int num_prog; int i; int j; int aux;
3     num_prog = HD_Read(0,0);
4     j = tab_inicio;
5     i = 0;
6     while(i < num_prog){
7         if(HD_Read(0,j) == 1){
8             i = i + 1;
9             aux = HD_Read(0,j+1);
10            if (aux == program){
11                return j;
12            }
13        }
14        j = j + tab_space_program;
15    }
16    return NOT_FOUND_LCD;
17 }
```

De acordo com o parâmetro *program*, a função *findProgram* verifica se existe algum programa com este nome. Caso exista, retorna a sua posição na tabela de alocação de arquivos. Caso contrário, é retornado o valor *NOT_FOUND_LCD*.

```

1  int loadProgram (int posMI, int program){
2      int pos; int start; int end; int i; int sector;
3      pos = findProgram(program);
4      if(pos == NOT_FOUND_LCD){
5          output(NOT_FOUND_LCD);
6          return 0;
7      }
8      if(pos == tab_inicio){
9          output(ERROR_LCD);
10         return 0;
11     }
12     start = HD_Read(0, pos+2);
13     end = start + HD_Read(0, pos+3);
14     sector = start/1024;
15     for(i = start; i < end; i = i + 1){
16         HD_Transfer_MI(posMI, sector, i - sector*1024);
17         posMI = posMI + 1;
18     }
19     return 1;
20 }

```

A função *loadProgram* carrega um programa na memória de instruções conforme o valor *posMI* passado como parâmetro.

1. O programa é procurado com a função *findProgram*;
2. Caso o programa não exista: imprime *NOT_FOUND_LCD* e retorna o valor zero;
3. Caso o programa exista, mas é o sistema operacional: imprime *ERROR_LCD* e retorna o valor zero.
4. Caso o programa exista e não é o SO, é carregado o programa na memória de instruções e é retornado o valor um.

Em seguida, será explicado o funcionamento de cada opção do *prompt* de comando do sistema operacional.

4.3.7.1 Executar um programa

Ao selecionar executar um programa, a mensagem *EXEC* é impressa no LCD para apresentar a opção escolhida. Posteriormente, deve-se entrar com o nome do programa a ser executado; este nome é passado como parâmetro para a função *executeProgram*.

```

1  void executeProgram(int program){
2      int posMI_S0; int posMD_S0; int so_location;
3      so_location = findProgram(0);
4      posMI_S0 = HD_Read(0, so_location + 3);
5      posMD_S0 = HD_Read(0, so_location + 4);
6      if(loadProgram(posMI_S0, program) == 1){
7          setNumProg(program);
8          output(0);
9          execProcess(0, posMI_S0, posMD_S0);

```

```
10 }  
11 }
```

A função *executeProgram* irá:

1. Procurar o sistema operacional para saber qual o espaço de memória utilizado por ele;
2. Carregar o programa na memória de instruções a partir da última posição de memória utilizada pelo SO;
3. Caso não ocorra erros ao carregar o programa:
 - a) Define o programa que está sendo executado;
 - b) Imprime o valor zero para apagar o último valor de saída;
 - c) Executa o programa a partir da primeira instrução. O espaço de memória utilizado pelo sistema operacional é passado como parâmetro da função *execProcess* para a realizar o deslocamento das memórias (instruções e dados).

4.3.7.2 Executar um conjunto de programas

Ao selecionar executar um conjunto de programas, a mensagem *EXEC_N* é impressa no LCD. Em seguida, a função *executeNPrograms* é chamada.

```
1 void executeNPrograms(){  
2     int i; int base; int sector;  
3     int so_location;  
4     int posMI; int posMD;  
5     int program; int prog_location;  
6     exec = 0;  
7     n_process_exec = input();  
8     if(n_process_exec < 2){  
9         output(ERROR_LCD);  
10        returnMain();  
11    }  
12    output(n_process_exec);  
13    qtd_processos_ativos = n_process_exec;  
14    so_location = findProgram(0);  
15    posMI = HD_Read(0, so_location + 3);  
16    posMD = HD_Read(0, so_location + 4);  
17    i=0; base=0;  
18    while(i<n_process_exec){  
19        program = input();  
20        if(program != 0){  
21            prog_location = findProgram(program);  
22            if(prog_location!=NOT_FOUND_LCD){  
23                i=i+1;  
24                if(loadProgram(posMI, program) == 1){  
25                    output(program);  
26                    output(OK_LCD);  
27                }  
28                CS[base] = 0;    //Contador de programa
```

```

29     CS[base+1] = HD_Read(0, prog_location + 1); //Nome do programa
30     CS[base+2] = 1; //Em execucao?
31     CS[base+3] = posMI; //Base Memoria de Instrucoes
32     CS[base+4] = posMD; //Base Memoria de Dados
33     CS[base+5] = HD_Read(0, prog_location + 3); //Quantidade de instrucoes
34     sector = (HD_Read(0, prog_location + 2))/1024;
35     CS[base+6] = sector; //Ultimo setor utilizado
36     CS[base+7] = HD_Read(0, prog_location + 3); //Ultima trilha utilizada
37     CS[base+8] = 1; //Primeira vez a ser executado?
38     base = base + space_to_process;
39     posMI = posMI + HD_Read(0, prog_location+3);
40     posMD = posMD + HD_Read(0, prog_location+4);
41 }
42 else output(NOT_FOUND_LCD);
43 }
44 else output(OTHER_NAME_LCD);
45 }
46 tam_vector_CS = base;
47 for(i=0; i<tam_vector_CS; i=i+space_to_process){
48     output(CS[i+1]);
49 }
50 setNumProg(CS[exec+1]);
51 output(0);
52 setMultiprog(1);
53 execProcess(CS[exec], CS[exec+3], CS[exec+4]);
54 }

```

A função *executeNPrograms* inicia a execução de um conjunto de programas com preempção. O passo a passo da execução é apresentado abaixo:

1. Entrar com o número de programas que serão executados;
2. Se o número de programas é menor que dois, então imprime a mensagem *ERROR* no LCD e volta para a *main*;
3. Caso contrário, é impresso a quantidade de programas que serão executados;
4. Entrar com o nome de um programa, caso o programa escolhido:
 - a) não exista: imprime a mensagem *NOT_FOUND*;
 - b) seja o sistema operacional: imprime a mensagem *OTHER_NAME*;
 - c) exista:
 - i. o programa é carregado, apresentando o nome do programa no LCD e a mensagem *OK*;
 - ii. são armazenadas as seguintes informações do programa no vetor *CS*: contador de programa, nome do programa, se está em execução, deslocamento da memória (instruções e dados), quantidade de instruções, setor e trilha (serão utilizados para salvar o banco de registradores), primeira vez a ser executado;

- iii. atualiza-se o valor das variáveis: *base* (próxima posição para escrita no vetor *CS*), *posMI* (próxima posição para carregar novos processos) e *posMD* (próxima posição da memória de dados que está livre).
5. Enquanto todos os programas não forem escolhidos, o passo 4 é repetido;
6. Imprimir os programas escolhidos;
7. Definir a multiprogramação como ativada;
8. Executar o primeiro processo do vetor;

Ao iniciar a execução do primeiro programa, o contador *contQ* é incrementado a cada subida de *clock*. Quando este contador atingir o valor do *quantum*, é executado a função *contextSwitch* que executa a troca de contexto.

```

1 void contextSwitch (){
2     int i; int stay; int one_process;
3     int pc_process; int prox_exec;
4     setMultiprog(0);
5     pc_process = getPC_Process();
6     stay = 0;
7     one_process = qtd_processos_ativos;
8     if(pc_process == (CS[exec+5]-1)){
9         CS[exec + 2] = 0;
10        qtd_processos_ativos = qtd_processos_ativos - 1;
11    }
12    else{
13        for(i = startRF; i <= endRF; i = i + 1){
14            saveRF_in_HD(i, CS[exec+6], CS[exec+7] + i - startRF);
15        }
16        CS[exec] = pc_process;
17    }
18    if(qtd_processos_ativos == 0){
19        setNumProg(0);
20        output(OK_LCD);
21        returnMain();
22    }
23    if(one_process != 1){
24        setNumProg(0);
25        output(CS_LCD);
26    }
27    for(i = 0; i < n_process_exec; i = i + 1){
28        if(stay == 0){
29            prox_exec = exec + space_to_process;
30            if(prox_exec >= tam_vector_CS) exec = 0;
31            else exec = prox_exec;
32            if(CS[exec+2] == 1)
33                stay = 1;
34            else stay = 0;
35        }
36    }
37    if(stay == 1){
38        setNumProg(CS[exec+1]);
39        if(CS[exec+8] == 1){
40            output(0);

```

```
41     CS[exec+8] = 0;
42 }
43 else{
44     for(i = startRF; i <= endRF; i = i+1){
45         recoveryRF(i, CS[exec+6], CS[exec+7] + i - startRF);
46     }
47     if(one_process != 1) delay();
48 }
49 setMultiprog(1);
50 execProcess(CS[exec], CS[exec+3], CS[exec+4]);
51 }
52 returnMain();
53 }
```

A função é *contextSwitch* irá:

- Definir a multiprogramação como zero: para não ocorrer a troca de contexto durante a execução do SO;
- Guardar o valor do contador de programa do processo que estava sendo executado;
- Verificar se o processo terminou, checando em qual instrução parou;
- Caso não tenha terminado a execução do processo, será salvo o banco de registradores no HD, além de salvar o contador de programa no vetor responsável pela troca de contexto.
- Verificar a quantidade processos ativos, caso já todos já tiverem terminado sua execução, é retornado para a *main*;
- Imprime a mensagem *CS* para indicar que está realizando a troca de contexto;
- Procura por um novo processo a ser executado;
- Caso seja a primeira vez que este processo é executado, é impresso o valor zero no LCD;
- Caso contrário, o banco de registradores é recuperado a partir do HD;
- É definido a multiprogramação como um e inicia a execução de um novo processo.

Como citado anteriormente, o banco de registradores foi dividido de acordo com a tabela 2. O espaço reservado para os processos é diferente do reservado para o sistema operacional. Desta forma, quando for realizar a troca de contexto, o SO não sobrescreverá o banco de registradores utilizado pelo processo; consequentemente, a função do sistema operacional conseguirá manter a consistência dos dados.

4.3.7.3 Renomear um programa

```

1  int renameProgram(int program){
2      int j; int num_prog; int new_name;
3      num_prog = HD_Read(0,0);
4      j = findProgram(program);
5      if(j == NOT_FOUND_LCD) return NOT_FOUND_LCD;
6      if(j == tab_inicio) return ERROR_LCD;
7      else {
8          new_name = input();
9          if(findProgram(new_name) == NOT_FOUND_LCD){
10             HD_Write(new_name, 0, j+1);
11             return OK_LCD;
12         }
13         else{
14             return OTHER_NAME_LCD;
15         }
16     }
17 }

```

A função *renameProgram* irá:

1. Procurar o programa que será renomeado. Caso este programa:
 - a) não exista: imprime a mensagem *NOT_FOUND*;
 - b) seja o sistema operacional: imprime a mensagem *ERROR_NAME*;
 - c) exista:
 - i. Deve-se escolher um novo nome para o programa;
 - ii. Caso seja um nome válido (ou seja, nenhum outro programa tenha este novo nome), é escrito na tabela de alocação de arquivos o novo nome do programa e é impresso a mensagem *OK*;
 - iii. Caso contrário, é impresso a mensagem *OTHER_NAME* indicando que o novo nome escolhido não pode ser escolhido.

4.3.7.4 Criar um programa

```

1  int createProgram(){
2      int num_prog; int prox_pos;
3      int instruction; int name; int i;
4      int sector; int track;
5      num_prog = HD_Read(0,0);
6      prox_pos = HD_Read(0,2);
7      i = tab_inicio;
8      while(HD_Read(0,i) == 1){
9          i = i + tab_space_program;
10     }
11     name = input();
12     while(findProgram(name) != NOT_FOUND_LCD) {
13         output(OTHER_NAME_LCD);
14         name = input();
15     }
16     output(0);
17     HD_Write(1, 0, i);

```

```
18  HD_Write(name, 0, i+1);
19  HD_Write(prox_pos, 0, i+2);
20  HD_Write(4, 0, i+3);
21  HD_Write(0, 0, i+4);
22  instruction = input();
23  instruction = instruction * 65536;
24  sector = prox_pos/1024;
25  track = prox_pos - sector*1024;
26  HD_Write(instruction, sector, track);
27  instruction = input();
28  instruction = instruction * 65536;
29  track = (prox_pos+1) - sector*1024;
30  HD_Write(instruction, sector, track);
31  instruction = input();
32  instruction = instruction * 65536;
33  track = (prox_pos+2) - sector*1024;
34  HD_Write(instruction, sector, track);
35  instruction = input();
36  instruction = instruction * 65536;
37  track = (prox_pos+3) - sector*1024;
38  HD_Write(instruction, sector, track);
39  HD_Write(num_prog+1, 0, 0);
40  HD_Write(prox_pos+1024, 0, 2);
41  return OK_LCD;
42 }
```

A função *createProgram* irá:

1. Atribuir a quantidade programas que existem no HD à variável *num_prog*;
2. Saber qual a próxima posição para escrita no HD (variável *prox_pos*);
3. Procurar pelo primeiro bloco da tabela de alocação de arquivos que não é utilizado;
4. Escolher o nome para o programa;
5. Caso o nome não seja válido, é impresso a mensagem *OTHER_NAME*;
6. Enquanto este nome não for válido, voltar ao passo 4;
7. É impresso o valor zero para indicar que o nome é válido;
8. É escrito na tabela de alocação de arquivos algumas informações do programa: espaço da tabela usado ou não, nome do programa, início do programa, quantidade de instruções e o tamanho da memória de dados.
9. Entrar com 4 instruções para o programa, estas instruções não devem usar a memória de dados e devem ser simples, ou seja, só devem utilizar até 16 *bits*.
10. Para cada instrução, o valor de entrada é deslocado 16 *bits* para a esquerda, multiplicando pelo valor 65536;
11. Por fim, tanto o número de programas quanto a próxima posição para escrita são atualizados no HD.

4.3.7.5 Deletar um programa

```
1 int deleteProgram(int program){
2     int j; int num_prog;
3     num_prog = HD_Read(0,0);
4     j = findProgram(program);
5     if(j == NOT_FOUND_LCD) return NOT_FOUND_LCD;
6     if(j == tab_inicio) return ERROR_LCD;
7     HD_Write(0, 0, j);
8     HD_Write(num_prog - 1, 0, 0);
9     return OK_LCD;
10 }
```

A função *deleteProgram* irá:

1. Atribuir a quantidade de programas à variável *num_prog*;
2. Procurar o programa na tabela de alocação de arquivos, caso o programa:
 - a) Não exista: imprime a mensagem *NOT_FOUND*;
 - b) Seja o sistema operacional: imprime a mensagem *ERROR*;
 - c) Exista: a referência do programa é apagada da tabela de alocação de arquivos e é atualizado a quantidade de programas.

4.3.7.6 Mostrar programas

```
1 void showPrograms(){
2     int num_prog; int space_prog; int i; int j; int aux;
3     num_prog = HD_Read(0,0);
4     j = tab_inicio;
5     i = 0;
6     while(i < num_prog){
7         if(HD_Read(0,j) == 1){
8             i = i + 1;
9             output(HD_Read(0,j+1));
10        }
11        j = j + tab_space_program;
12    }
13 }
```

A função *showPrograms* irá:

1. Atribuir a quantidade de programas à variável *num_prog*;
2. Percorre a tabela de alocação de arquivos, apresentando todos os programas que estão no HD.

5 Resultados Obtidos e Discussões

Nesta parte do relatório, será apresentado os resultados obtidos com a execução do sistema operacional. Isto irá incluir o funcionamento da BIOS e do *prompt* de comando.

5.1 Funcionamento da BIOS

De acordo com o teste do LCD, deve ser apresentado o valor três no LCD. Como pode ser observado na figura 7, este é o valor que se esperava. Desta forma, este componente está funcionando normalmente.

Figura 7 – Teste do LCD.



A figura 8 demonstra o funcionamento da entrada de dados. O valor de entrada inserido foi o valor doze e o valor apresentado no LCD é exatamente o mesmo.



Figura 8 – Teste da entrada de dados.

O teste da memória de dados é composto por guardar o valor quatro na memória e apresentá-lo no LCD. O valor apresentado na figura 9 é o esperado.



Figura 9 – Teste da memória de dados.

Para testar o processador, é realizada uma soma entre o valor três e o valor cinco. E o valor apresentado na figura 10 é oito como esperado.



Figura 10 – Teste do processador.

Para a identificar o dispositivo de armazenamento principal, é realizado a escrita do valor dez no HD que, posteriormente, é lido e apresentado no LCD. Como pode ser observado na figura 11, o valor dez é apresentado na saída.



Figura 11 – Identificação do dispositivo de armazenamento principal.

Como citado anteriormente, para cada um dos testes apresentados, é necessário confirmar a corretude do valor de saída. Caso o valor esteja correto, deve-se entrar com o valor um; caso contrário, deve-se entrar com o valor zero. A figura 12 apresenta a mensagem de erro. Já a figura 13 mostra a mensagem de funcionamento correto do dispositivo.



Figura 12 – Apresenta o erro na execução da BIOS.



Figura 13 – Apresenta que a BIOS foi executada de forma correta.

Por fim, o processador carrega o sistema operacional para a memória principal e deve apresentar a mensagem "OK". Como é mostrado na figura 14, este é o resultado apresentado no LCD.



Figura 14 – Resultado da transferência do sistema operacional para a memória de instruções.

Portanto, a BIOS foi executada de forma correta.

5.2 Funcionamento do prompt de comando

A seguir será apresentado o funcionamento do *prompt* de comando. Para cada função do *prompt* será realizado um teste e demonstrado o seu funcionamento.

5.2.1 Execução de um programa específico

A figura 15 apresenta que foi escolhido a opção da execução de um programa em específico. O programa escolhido para execução foi o programa número um (busca binária B.1) e entrada para este processo foi o valor oito. Como este valor está localizado na posição dois, este deve ser o valor de saída. E observando a figura 16, a execução de um programa específico está funcionando normalmente.

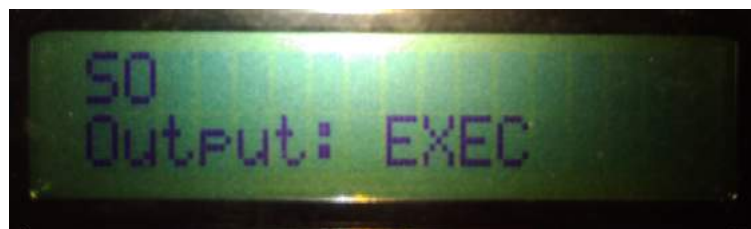


Figura 15 – Opção para execução de um programa específico.

5.2.2 Execução de um conjunto de programas

A figura 17 apresenta a opção de execução de um conjunto de programas. Para este teste, foi escolhido dois programas para serem executados: o fatorial (B.2) e o fibonacci



Figura 16 – Resultado do processo.

(B.3). Para os dois programas o valor de entrada foi o valor quatro. A figura 18 apresenta a execução da troca de contexto dos processos ao atingir o *quantum* do *Round-Robin*.

O fatorial de 4 é 24 (figura 19). E o fibonacci de 4 é 3 (figura 20). Assim, o teste para a execução de um conjunto de programas está funcionando.



Figura 17 – Opção para execução de um conjunto de programas.



Figura 18 – Troca de contexto sendo realizada.



Figura 19 – Resultado processo P2.



Figura 20 – Resultado processo P3.

5.2.3 Renomear um programa

A figura 21 apresenta que foi escolhido a opção de renomear um programa. Neste teste, foi renomeado o programa número 3 para o número 11; o resultado é apresentado na figura 22, o que indica o sucesso da operação.



Figura 21 – Opção para renomear um programa.



Figura 22 – Resultado da operação de renomear um programa.

5.2.4 Criar um programa

Ao escolher a criação de um programa, o LCD mostra a saída apresentada na figura 23. Posteriormente, entrou-se com o valor 12 para o novo nome do programa e com as instruções pelas chaves. O programa criado realiza somente uma leitura e uma saída de dados. Por fim, imprimiu a mensagem de sucesso (figura 24).



Figura 23 – Opção para criar um programa.



Figura 24 – Resultado da operação para criação do programa.

5.2.5 Deletar um programa

A figura 21 apresenta que foi escolhido a opção de deletar um programa. Foi escolhido o programa 8 para ser deletado, o resultado é apresentado na figura 26 o que indica sucesso na operação.



Figura 25 – Opção para deletar um programa.



Figura 26 – Resultado da operação de deletar um programa.

5.2.6 Mostrar programas

A figura 27 apresenta que foi escolhido a opção de mostrar os programas. Ao executar tal função, o sistema operacional apresentou todas os programas atuais. Esta parte não será apresentada, pois são muitas imagens e com pouca informação.



Figura 27 – Opção para mostrar programas.

5.2.7 Mensagens de erros

Por fim, serão apresentados as mensagens de erros que podem aparecer ao executar o sistema operacional.

A figura 28 expõe o erro que aconteceu no teste da execução de um programa, foi escolhido o sistema operacional para ser executado. Já o erro da figura 29 ocorreu quando foi selecionado um programa que não existe para ser executado. Por último, a figura 30 mostra o erro quando tentou-se renomear um programa, o novo nome escolhido já era utilizado por outro programa.



Figura 28 – Mensagem de erro: erro.



Figura 29 – Mensagem de erro: programa não encontrado.



Figura 30 – Mensagem de erro: escolher outro nome.

6 Considerações Finais

O sistema computacional desenvolvido neste trabalho foi composto de uma plataforma de *hardware*, um compilador, uma BIOS e o código do sistema operacional (SO). A plataforma de *hardware* foi baseada na arquitetura MIPS, esta plataforma foi alterada de forma a ser capaz de executar códigos com preempção; foram adicionadas instruções com teste objetivo. O compilador é responsável por converter o código *C*- em código de máquina, além de iniciar o gerenciamento do sistema de arquivos. A BIOS realiza o teste de partida, verificando os componentes da plataforma de *hardware*, transfere as instruções do sistema operacional do HD para a memória de instruções e inicia a execução do SO.

De acordo com as tarefas do *prompt* de comando, o sistema operacional realiza o gerenciamento de processos, de memória, de arquivos e de entrada/saída. Com relação ao gerenciador de processos, foi utilizado o algoritmo de *Round-Robin* que utiliza o valor do *quantum* para a preempção. Para o gerenciamento de memória, foi empregado um deslocamento na memória (de dados e de instruções) para a execução de cada processo. Quanto ao gerenciador de arquivos, a tabela de alocação de arquivos é iniciada pelo compilador e atualizada pelo sistema operacional. E, por último, o gerenciador de entrada/saída é realizado de acordo com o *clock*.

Este *prompt* de comando tem como opções: executar um programa específico, executar um conjunto de programas, renomear um programa, criar um programa, deletar um programa e mostrar os programas armazenados no HD. Todas essas opções foram testadas e apresentaram sucesso em sua execução.

O desenvolvimento deste sistema computacional não foi simples, houveram diversas dificuldades. Como o entendimento do funcionamento da troca de contexto de acordo com o processador e com o compilador que foram desenvolvidos; a ideia inicial para solução deste problema foi adotada do início ao fim, porém, houveram alguns impasses durante este processo. Um deles foi desenvolver soluções que não abrangiam todos os casos. Outro impasse foi com relação ao *software* Quartus que travava durante a compilação da plataforma de *hardware*, o problema estava na utilização das operações de multiplicação e divisão da própria linguagem de descrição de *hardware* verilog. Ao excluir estas operações do código, a compilação voltou a ocorrer sem erros.

Como próximos passos, pretende-se implementar a interrupção na plataforma de *hardware*. A interrupção será bastante útil para o laboratório de Redes de Computadores, visto que pode ser necessário trocar a execução de processos ao receber um sinal externo.

Referências

- DELL. *No prompt de comando: o que é e como usá-lo em um sistema Dell*. 2017. <<http://www.dell.com/support/article/br/pt/brbsdt1/sln265940/>>. Acessado em 21 mar. 2018. Citado na página 14.
- DELL. *BIOS: O que é e como fazer download ou atualizar o seu BIOS*. 2018. <<http://www.dell.com/support/article/br/pt/brbsdt1/sln129956/>>. Acessado em 21 mar. 2018. Citado na página 14.
- LOOMIS, J. *Digital Labs using the Altera DE2 Board*. 2008. <<http://www.johnloomis.org/digitallab/>>. Acessado em 29 mar. 2018. Citado na página 20.
- LOUDEN, K. *Compiladores: Princípios e Práticas*. 1ª edição. ed. São Paulo/SP, BR: Thomson, 2004. Citado na página 12.
- PASSOS, D. *UCP: Instrução Jump, Monociclo vs. Multiciclo, Pipeline*. 2015. <http://www.midiacom.uff.br/~diego/disciplinas/2015_1/FAC/arquivos/aula21.pdf>. Acessado em 21 mar. 2018. Citado na página 12.
- PATTERSON, D.; HENNESSY, J. *Organização e projeto de computadores: a interface hardware / software*. 3ª edição. ed. Rio de Janeiro/RJ, BR: Campus, 2005. Citado na página 11.
- RICARTE, I. *Compiladores*. 2003. <<http://www.dca.fee.unicamp.br/cursos/EA876/apostila/HTML/>>. Acessado em 24 mar. 2017. Citado 2 vezes nas páginas 12 e 13.
- STOREY, T. *Debounce Logic Circuit (with Verilog example)*. 2013. <<https://eewiki.net/pages/viewpage.action?pageId=13599139>>. Acessado em 29 mar. 2018. Citado na página 20.
- TANENBAUM, A. S. *Sistemas Operacionais Modernos*. 3ª edição. ed. São Paulo/SP, BR: Pearson Prentice Hall, 2009. Citado 5 vezes nas páginas 14, 15, 16, 17 e 18.
- TORRES, G. *Sistema de arquivos*. 1997. <<https://www.clubedohardware.com.br/artigos/armazenamento/sistema-de-arquivos-r33864/>>. Acessado em 22 mar. 2018. Citado na página 18.

Apêndices

APÊNDICE A – Código fonte

A.1 BIOS.cm

```
1  int ERRO;  
2  int OK_LCD;  
3  int ERROR_LCD;  
4  
5  void confirmacao(){  
6      int conf;  
7      conf = input();  
8      if(conf == 1){  
9          output(OK_LCD);  
10         ERRO = 0;  
11     }  
12     else output(ERROR_LCD);  
13 }  
14  
15 void LED(){  
16     output(3);  
17     confirmacao();  
18 }  
19  
20 void entrada(){  
21     int n;  
22     n = input();  
23     output(n);  
24     confirmacao();  
25 }  
26  
27 void memoriaDeDados(){  
28     int n;  
29     n = 4;  
30     output(n);  
31     confirmacao();  
32 }  
33  
34 void processador(){  
35     output(3+5);  
36     confirmacao();  
37 }  
38  
39 void hd_test(){  
40     HD_Write(10, 15, 1023);  
41     output(HD_Read(15, 1023));  
42     confirmacao();  
43 }  
44  
45 void hd_instr(){  
46     int i; int j;  
47     j = 0;  
48     for(i = 0; i < 2048; i = i+1){  
49         if(i < 1024)  
50             HD_Transfer_MI(i, 1, i);  
51         else{
```

```

52     HD_Transfer_MI(i, 2, j);
53     j = j + 1;
54 }
55 }
56 }
57
58 void main(void){
59     setMultiprog(0);
60     delay();
61     OK_LCD = 65535;
62     ERROR_LCD = 65534;
63     ERRO = 1;
64     while(ERRO == 1) LED();
65     ERRO = 1;
66     while(ERRO == 1) entrada();
67     ERRO = 1;
68     while(ERRO == 1) memoriaDeDados();
69     ERRO = 1;
70     while(ERRO == 1) processador();
71     ERRO = 1;
72     while(ERRO == 1) hd_test();
73     hd_instr();
74     output(OK_LCD);
75 }

```

A.2 SO.cm

```

1  int OK_LCD;
2  int ERROR_LCD;
3  int OPTION_LCD;
4  int CS_LCD;
5  int EXECUTE_LCD;
6  int EXECUTE_N_LCD;
7  int RENAME_LCD;
8  int CREATE_LCD;
9  int DELETE_LCD;
10 int SHOW_LCD;
11 int OTHER_NAME_LCD;
12 int NOT_FOUND_LCD;
13 int tab_inicio; int tab_space_program;
14 int exec; int space_to_process; int qtd_processos_ativos;
15 int tam_vector_CS; int n_process_exec; int CS[100];
16 int startRF; int endRF;
17
18 void contextSwitch (){
19     int i; int stay; int one_process;
20     int pc_process; int prox_exec;
21     setMultiprog(0);
22     pc_process = getPC_Process();
23     stay = 0;
24     one_process = qtd_processos_ativos;
25     if(pc_process == (CS[exec+5]-1)){
26         CS[exec + 2] = 0;
27         qtd_processos_ativos = qtd_processos_ativos - 1;
28     }
29     else{
30         for(i = startRF; i <= endRF; i = i + 1){
31             saveRF_in_HD(i, CS[exec+6], CS[exec+7] + i - startRF);
32         }

```

```

33     CS[exec] = pc_process;
34 }
35 if(qtd_processos_ativos == 0){
36     setNumProg(0);
37     output(OK_LCD);
38     returnMain();
39 }
40 if(one_process != 1){
41     setNumProg(0);
42     output(CS_LCD);
43 }
44 for(i = 0; i < n_process_exec; i = i + 1){
45     if(stay == 0){
46         prox_exec = exec + space_to_process;
47         if(prox_exec >= tam_vector_CS) exec = 0;
48         else exec = prox_exec;
49         if(CS[exec+2] == 1)
50             stay = 1;
51         else stay = 0;
52     }
53 }
54 if(stay == 1){
55     setNumProg(CS[exec+1]);
56     if(CS[exec+8] == 1){
57         output(0);
58         CS[exec+8] = 0;
59     }
60     else{
61         for(i = startRF; i <= endRF; i = i+1){
62             recoveryRF(i, CS[exec+6], CS[exec+7] + i - startRF);
63         }
64         if(one_process != 1) delay();
65     }
66     setMultiprog(1);
67     execProcess(CS[exec], CS[exec+3], CS[exec+4]);
68 }
69 returnMain();
70 }
71
72 int findProgram(int program){
73     int num_prog; int i; int j; int aux;
74     num_prog = HD_Read(0,0);
75     j = tab_inicio;
76     i = 0;
77     while(i < num_prog){
78         if(HD_Read(0,j) == 1){
79             i = i + 1;
80             aux = HD_Read(0,j+1);
81             if (aux == program){
82                 return j;
83             }
84         }
85         j = j + tab_space_program;
86     }
87     return NOT_FOUND_LCD;
88 }
89
90 int loadProgram (int posMI, int program){
91     int pos; int start; int end; int i; int sector;
92     pos = findProgram(program);

```

```

93     if(pos == NOT_FOUND_LCD){
94         output(NOT_FOUND_LCD);
95         return 0;
96     }
97     if(pos == tab_inicio){
98         output(ERROR_LCD);
99         return 0;
100    }
101    start = HD_Read(0, pos+2);
102    end = start + HD_Read(0, pos+3);
103    sector = start/1024;
104    for(i = start; i < end; i = i + 1){
105        HD_Transfer_MI(posMI, sector, i - sector*1024);
106        posMI = posMI + 1;
107    }
108    return 1;
109 }
110
111 void executeNPrograms(){
112     int i; int base; int sector;
113     int so_location;
114     int posMI; int posMD;
115     int program; int prog_location;
116     exec = 0;
117     n_process_exec = input();
118     if(n_process_exec < 2){
119         output(ERROR_LCD);
120         returnMain();
121     }
122     output(n_process_exec);
123     qtd_processos_ativos = n_process_exec;
124     so_location = findProgram(0);
125     posMI = HD_Read(0, so_location + 3);
126     posMD = HD_Read(0, so_location + 4);
127     i=0; base=0;
128     while(i<n_process_exec){
129         program = input();
130         if(program != 0){
131             prog_location = findProgram(program);
132             if(prog_location!=NOT_FOUND_LCD){
133                 i=i+1;
134                 if(loadProgram(posMI, program) == 1){
135                     output(program);
136                     output(OK_LCD);
137                 }
138                 CS[base] = 0;
139                 CS[base+1] = HD_Read(0, prog_location + 1);
140                 CS[base+2] = 1;
141                 CS[base+3] = posMI;
142                 CS[base+4] = posMD;
143                 CS[base+5] = HD_Read(0, prog_location + 3);
144                 sector = (HD_Read(0, prog_location + 2))/1024;
145                 CS[base+6] = sector;
146                 CS[base+7] = HD_Read(0, prog_location + 3);
147                 CS[base+8] = 1;
148                 base = base + space_to_process;
149                 posMI = posMI + HD_Read(0, prog_location+3);
150                 posMD = posMD + HD_Read(0, prog_location+4);
151             }
152             else output(NOT_FOUND_LCD);

```



```

153     }
154     else output(OTHER_NAME_LCD);
155 }
156 tam_vector_CS = base;
157 for(i=0; i<tam_vector_CS; i=i+space_to_process){
158     output(CS[i+1]);
159 }
160 setNumProg(CS[exec+1]);
161 output(0);
162 setMultiprog(1);
163 execProcess(CS[exec], CS[exec+3], CS[exec+4]);
164 }
165
166 void executeProgram(int program){
167     int posMI_S0; int posMD_S0; int so_location;
168     so_location = findProgram(0);
169     posMI_S0 = HD_Read(0, so_location + 3);
170     posMD_S0 = HD_Read(0, so_location + 4);
171     if(loadProgram(posMI_S0, program) == 1){
172         setNumProg(program);
173         output(0);
174         execProcess(0, posMI_S0, posMD_S0);
175     }
176 }
177
178 int createProgram(){
179     int num_prog; int prox_pos;
180     int instruction; int name; int i;
181     int sector; int track;
182     num_prog = HD_Read(0,0);
183     prox_pos = HD_Read(0,2);
184     i = tab_inicio;
185     while(HD_Read(0,i) == 1){
186         i = i + tab_space_program;
187     }
188     name = input();
189     while(findProgram(name) != NOT_FOUND_LCD) {
190         output(OTHER_NAME_LCD);
191         name = input();
192     }
193     output(0);
194     HD_Write(1, 0, i);
195     HD_Write(name, 0, i+1);
196     HD_Write(prox_pos, 0, i+2);
197     HD_Write(4, 0, i+3);
198     HD_Write(0, 0, i+4);
199     instruction = input();
200     instruction = instruction * 65536;
201     sector = prox_pos/1024;
202     track = prox_pos - sector*1024;
203     HD_Write(instruction, sector, track);
204     instruction = input();
205     instruction = instruction * 65536;
206     track = (prox_pos+1) - sector*1024;
207     HD_Write(instruction, sector, track);
208     instruction = input();
209     instruction = instruction * 65536;
210     track = (prox_pos+2) - sector*1024;
211     HD_Write(instruction, sector, track);
212     instruction = input();

```

```

213     instruction = instruction * 65536;
214     track = (prox_pos+3) - sector*1024;
215     HD_Write(instruction, sector, track);
216     HD_Write(num_prog+1, 0, 0);
217     HD_Write(prox_pos+1024, 0, 2);
218     return OK_LCD;
219 }
220
221 void showPrograms(){
222     int num_prog; int space_prog; int i; int j; int aux;
223     num_prog = HD_Read(0,0);
224     j = tab_inicio;
225     i = 0;
226     while(i < num_prog){
227         if(HD_Read(0,j) == 1){
228             i = i + 1;
229             output(HD_Read(0,j+1));
230         }
231         j = j + tab_space_program;
232     }
233 }
234
235 int deleteProgram(int program){
236     int j; int num_prog;
237     num_prog = HD_Read(0,0);
238     j = findProgram(program);
239     if(j == NOT_FOUND_LCD) return NOT_FOUND_LCD;
240     if(j == tab_inicio) return ERROR_LCD;
241     HD_Write(0, 0, j);
242     HD_Write(num_prog - 1, 0, 0);
243     return OK_LCD;
244 }
245
246 int renameProgram(int program){
247     int j; int num_prog; int new_name;
248     num_prog = HD_Read(0,0);
249     j = findProgram(program);
250     if(j == NOT_FOUND_LCD) return NOT_FOUND_LCD;
251     if(j == tab_inicio) return ERROR_LCD;
252     else {
253         new_name = input();
254         if(findProgram(new_name) == NOT_FOUND_LCD){
255             HD_Write(new_name, 0, j+1);
256             return OK_LCD;
257         }
258         else{
259             return OTHER_NAME_LCD;
260         }
261     }
262 }
263
264 void main(){
265     int option; int program;
266     setMultiprogram(0);
267     OK_LCD = 65535; ERROR_LCD = 65534; OPTION_LCD = 65533; CS_LCD = 65532;
268     EXECUTE_LCD = 65531; EXECUTE_N_LCD = 65530; RENAME_LCD = 65529;
269     CREATE_LCD = 65528; DELETE_LCD = 65527; SHOW_LCD = 65526;
270     OTHER_NAME_LCD = 65525; NOT_FOUND_LCD = 65524;
271     startRF = 6; endRF = 19;
272     setNumProg(0);

```

```
273  setAddrCS(1);
274  setQuantum(50);
275  tab_inicio = 3; tab_space_program = 5;
276  space_to_process = 9;
277  while (1 == 1){
278      output(OPTION_LCD);
279      option = input();
280      if(option == 1){
281          output(EXECUTE_LCD);
282          program = input();
283          executeProgram(program);
284      }
285      else if(option == 2){
286          output(EXECUTE_N_LCD);
287          executeNPrograms();
288      }
289      else if(option == 3){
290          output(RENAME_LCD);
291          program = input();
292          output(program);
293          output(renameProgram(program));
294      }
295      else if(option == 4){
296          output(CREATE_LCD);
297          output(createProgram());
298      }
299      else if(option == 5){
300          output(DELETE_LCD);
301          program = input();
302          output(deleteProgram(program));
303      }
304      else if(option == 6){
305          output(SHOW_LCD);
306          showPrograms();
307      }
308  }
309 }
```


APÊNDICE B – Algoritmos para testes

B.1 *buscaBinaria.cm*

```
1 int buscaBinaria(int v[], int n, int x){
2     int e;
3     int m;
4     int d;
5     e = 0;
6     d = n-1;
7     while (e <= d) {
8         m = (e + d)/2;
9         if (v[m] == x) return m;
10        if (v[m] < x) e = m + 1;
11        else d = m - 1;
12    }
13    return 255;
14 }
15
16 void main(void){
17     int vetor[5];
18     int x;
19     vetor[0] = 0;
20     vetor[1] = 4;
21     vetor[2] = 8;
22     vetor[3] = 16;
23     vetor[4] = 32;
24     x = input();
25     output(buscaBinaria(vetor, 5, x));
26 }
```

B.2 *fatorial.cm*

```
1 int fatorial (int num){
2     int cont;
3     int resultado;
4     cont = 1;
5     resultado = 1;
6     while(cont <= num){
7         resultado = resultado * cont;
8         cont = cont + 1;
9     }
10    return resultado;
11 }
12
13 void main(void){
14     int n;
15     n = input();
16     n = fatorial(n);
17     output(n);
18 }
```

B.3 *fibonacci.cm*

```
1  int fibonacci(int n){
2      int c;
3      int next;
4      int first;
5      int second;
6      first = 0;
7      second = 1;
8      c = 0;
9      while(c <= n){
10         if(c <= 1)
11             next = c;
12         else{
13             next = first + second;
14             first = second;
15             second = next;
16         }
17         c = c + 1;
18     }
19     return next;
20 }
21 void main(void){
22     int n;
23     n = input();
24     output(fibonacci(n));
25 }
```

B.4 *mdc.cm*

```
1  int resto(int a, int b){
2      int c;
3      c = a/b;
4      return (a-b*c);
5  }
6
7  int mdc(int a, int b){
8      int r;
9      while (b != 0){
10         r = resto(a, b);
11         a = b;
12         b = r;
13     }
14     return a;
15 }
16
17 void main(void){
18     int numUM;
19     int numDOIS;
20     numUM = 10;
21     numDOIS = input();
22     output(mdc(numUM, numDOIS));
23 }
```

B.5 *media.cm*

```
1  int media (int v[], int tam){
```

```
2     int soma;
3     int aux;
4     aux = 0;
5     soma = 0;
6     while(aux < tam){
7         soma = soma + v[aux];
8         aux = aux + 1;
9     }
10    return soma/tam;
11 }
12 void main(void){
13     int v[5];
14     v[0] = 0;
15     v[1] = 1;
16     v[2] = 2;
17     v[3] = 3;
18     v[4] = input();
19     output(media(v, 5));
20 }
```

B.6 *mediana.cm*

```
1 void sort(int num[], int tam){
2     int i;
3     int j;
4     int min;
5     int aux;
6     i = 0;
7     while (i < tam-1){
8         min = i;
9         j = i + 1;
10        while (j < tam){
11            if(num[j] < num[min])
12                min = j;
13            j = j + 1;
14        }
15        if (i != min){
16            aux = num[i];
17            num[i] = num[min];
18            num[min] = aux;
19        }
20        i = i + 1;
21    }
22 }
23
24 int mediana(int v[], int tam){
25     return v[tam/2];
26 }
27
28 void main(void){
29     int vetor[5];
30     vetor[0] = 7;
31     vetor[1] = 0;
32     vetor[2] = 9;
33     vetor[3] = 10;
34     vetor[4] = input();
35     sort(vetor, 5);
36     output(mediana(vetor, 5));
37 }
```

B.7 mmc.cm

```
1  int resto(int a, int b){
2      int c;
3      c = a/b;
4      return (a-b*c);
5  }
6
7  int mdc(int a, int b){
8      int r;
9      while (b != 0){
10         r = resto(a, b);
11         a = b;
12         b = r;
13     }
14     return a;
15 }
16
17
18 int mmc(int a, int b){
19     int c;
20     c = (a*b);
21     c = c/mdc(a,b);
22     return c;
23 }
24
25 void main(void){
26     int numUM;
27     int numDOIS;
28     numUM = 10;
29     numDOIS = input();
30     output(mmc(numUM, numDOIS));
31 }
```

B.8 palindromo.cm

```
1  int resto(int a, int b){
2      int c;
3      c = a/b;
4      return (a-b*c);
5  }
6
7  int preencherVetor(int vetor[], int num){
8      int cont;
9      cont = 0;
10     while(num > 0){
11         vetor[cont] = resto(num, 10);
12         num = num/10;
13         cont = cont + 1;
14     }
15     return cont;
16 }
17
18 int palindromo (int num){
19     int vetor[5];
20     int i;
21     int tam;
22     int ehPalindromo;
23     i = 0;
```



```
24     ehPalindromo = 1;
25     tam = preencherVetor(vetor, num);
26     while(i < tam){
27         if(vetor[i] != vetor[tam-1-i])
28             ehPalindromo = 0;
29         i = i + 1;
30     }
31     return ehPalindromo;
32 }
33
34 void main(void){
35     int n;
36     n = input();
37     output(palindromo(n));
38 }
```

B.9 *potencia.cm*

```
1 int potencia (int num, int expoente){
2     int cont;
3     int resultado;
4     cont = 0;
5     resultado = 1;
6     while(cont < expoente){
7         resultado = resultado * num;
8         cont = cont + 1;
9     }
10    return resultado;
11 }
12
13 void main(void){
14     int n;
15     n = input();
16     output(potencia(n,2));
17 }
```

B.10 *selectionSort.cm*

```
1 void sort(int num[], int tam){
2     int i;
3     int j;
4     int min;
5     int aux;
6     i = 0;
7     while (i < tam-1){
8         min = i;
9         j = i + 1;
10        while (j < tam){
11            if(num[j] < num[min])
12                min = j;
13            j = j + 1;
14        }
15        if (i != min){
16            aux = num[i];
17            num[i] = num[min];
18            num[min] = aux;
19        }
20    }
```

```
20     i = i + 1;
21 }
22 }
23
24 void main(void){
25     int vetor[4];
26     int i;
27     vetor[0] = 9;
28     vetor[1] = 6;
29     vetor[2] = 8;
30     vetor[3] = 7;
31     sort(vetor, 4);
32     i = input();
33     output(vetor[i]);
34 }
```

Anexos

ANEXO A – Código para o LCD

O código para utilização do LCD foi desenvolvido pelo professor John Loomis e é apresentado abaixo sem as alterações realizadas.

A.1 lcdlab3.v

```

1 module lcdlab3(
2   input CLOCK_50,      // 50 MHz clock
3   input [3:0] KEY,      // Pushbutton[3:0]
4   input [17:0] SW,      // Toggle Switch[17:0]
5   output [6:0] HEX0,HEX1,HEX2,HEX3,HEX4,HEX5,HEX6,HEX7, // Seven Segment Digits
6   output [8:0] LEDG,    // LED Green
7   output [17:0] LEDR,   // LED Red
8   inout [35:0] GPIO_0,GPIO_1, // GPIO Connections
9   // LCD Module 16X2
10  output LCD_ON,        // LCD Power ON/OFF
11  output LCD_BLON,      // LCD Back Light ON/OFF
12  output LCD_RW,        // LCD Read/Write Select, 0 = Write, 1 = Read
13  output LCD_EN,        // LCD Enable
14  output LCD_RS,        // LCD Command/Data Select, 0 = Command, 1 = Data
15  inout [7:0] LCD_DATA  // LCD Data bus 8 bits
16 );
17
18 // All inout port turn to tri-state
19 assign GPIO_0 = 36'hzzzzzzzz;
20 assign GPIO_1 = 36'hzzzzzzzz;
21
22 wire [6:0] myclock;
23 wire RST;
24 assign RST = KEY[0];
25
26 // reset delay gives some time for peripherals to initialize
27 wire DLY_RST;
28 Reset_Delay r0( .iCLK(CLOCK_50),.oRESET(DLY_RST) );
29
30 // Send switches to red leds
31 assign LEDR = SW;
32
33 // turn LCD ON
34 assign LCD_ON = 1'b1;
35 assign LCD_BLON = 1'b1;
36
37 wire [3:0] hex1, hex0;
38 assign hex1 = SW[7:4];
39 assign hex0 = SW[3:0];
40
41
42 LCD_Display u1(
43 // Host Side
44 .iCLK_50MHZ(CLOCK_50),
45 .iRST_N(DLY_RST),
46 .hex0(hex0),
47 .hex1(hex1),

```

```

48 // LCD Side
49 .DATA_BUS(LCD_DATA),
50 .LCD_RW(LCD_RW),
51 .LCD_E(LCD_EN),
52 .LCD_RS(LCD_RS)
53 );
54
55
56 // blank unused 7-segment digits
57 assign HEX0 = 7'b111_1111;
58 assign HEX1 = 7'b111_1111;
59 assign HEX2 = 7'b111_1111;
60 assign HEX3 = 7'b111_1111;
61 assign HEX4 = 7'b111_1111;
62 assign HEX5 = 7'b111_1111;
63 assign HEX6 = 7'b111_1111;
64 assign HEX7 = 7'b111_1111;
65
66 endmodule

```

A.2 LCD_Display.v

```

1  /*
2   SW8 (GLOBAL RESET) resets LCD
3  ENTITY LCD_Display IS
4   -- Enter number of live Hex hardware data values to display
5   -- (do not count ASCII character constants)
6   GENERIC(Num_Hex_Digits: Integer:= 2);
7   -----
8   -- LCD Displays 16 Characters on 2 lines
9   -- LCD_display string is an ASCII character string entered in hex for
10  -- the two lines of the LCD Display (See ASCII to hex table below)
11  -- Edit LCD_Display_String entries above to modify display
12  -- Enter the ASCII character's 2 hex digit equivalent value
13  -- (see table below for ASCII hex values)
14  -- To display character assign ASCII value to LCD_display_string(x)
15  -- To skip a character use 8'h20" (ASCII space)
16  -- To display "live" hex values from hardware on LCD use the following:
17  -- make array element for that character location 8'h0" & 4-bit field from
   Hex_Display_Data
18  -- state machine sees 8'h0" in high 4-bits & grabs the next lower 4-bits from
   Hex_Display_Data input
19  -- and performs 4-bit binary to ASCII conversion needed to print a hex digit
20  -- Num_Hex_Digits must be set to the count of hex data characters (ie. "00"s) in the
   display
21  -- Connect hardware bits to display to Hex_Display_Data input
22  -- To display less than 32 characters, terminate string with an entry of 8'hFE"
23  -- (fewer characters may slightly increase the LCD's data update rate)
24  -----
25  --
26  --
27  --
28  --
29  --
30  --
31  --
32  --
33  --
34  --

```

ASCII HEX TABLE																
Hex	Low Hex Digit															
Value	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
--H 2	SP	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/
--i 3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
--g 4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
--h 5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
-- 6	'	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
-- 7	p	q	r	s	t	u	v	w	x	y	z	{		}	~	DEL

```

35 -----
36 -- Example "A" is row 4 column 1, so hex value is 8'h41"
37 -- *see LCD Controller's Datasheet for other graphics characters available
38 */
39
40 module LCD_Display(iCLK_50MHZ, iRST_N, hex1, hex0,
41     LCD_RS,LCD_E,LCD_RW,DATA_BUS);
42 input iCLK_50MHZ, iRST_N;
43 input [3:0] hex1, hex0;
44 output LCD_RS, LCD_E, LCD_RW;
45 inout [7:0] DATA_BUS;
46
47 parameter
48 HOLD = 4'h0,
49 FUNC_SET = 4'h1,
50 DISPLAY_ON = 4'h2,
51 MODE_SET = 4'h3,
52 Print_String = 4'h4,
53 LINE2 = 4'h5,
54 RETURN_HOME = 4'h6,
55 DROP_LCD_E = 4'h7,
56 RESET1 = 4'h8,
57 RESET2 = 4'h9,
58 RESET3 = 4'ha,
59 DISPLAY_OFF = 4'hb,
60 DISPLAY_CLEAR = 4'hc;
61
62 reg [3:0] state, next_command;
63 // Enter new ASCII hex data above for LCD Display
64 reg [7:0] DATA_BUS_VALUE;
65 wire [7:0] Next_Char;
66 reg [19:0] CLK_COUNT_400HZ;
67 reg [4:0] CHAR_COUNT;
68 reg CLK_400HZ, LCD_RW_INT, LCD_E, LCD_RS;
69
70 // BIDIRECTIONAL TRI STATE LCD DATA BUS
71 assign DATA_BUS = (LCD_RW_INT? 8'bZZZZZZZZ: DATA_BUS_VALUE);
72
73 LCD_display_string u1(
74     .index(CHAR_COUNT),
75     .out(Next_Char),
76     .hex1(hex1),
77     .hex0(hex0));
78
79 assign LCD_RW = LCD_RW_INT;
80
81 always @(posedge iCLK_50MHZ or negedge iRST_N)
82     if (!iRST_N)
83         begin
84             CLK_COUNT_400HZ <= 20'h00000;
85             CLK_400HZ <= 1'b0;
86         end
87     else if (CLK_COUNT_400HZ < 20'h0F424)
88         begin
89             CLK_COUNT_400HZ <= CLK_COUNT_400HZ + 1'b1;
90         end
91     else
92         begin
93             CLK_COUNT_400HZ <= 20'h00000;
94             CLK_400HZ <= ~CLK_400HZ;

```

```

95     end
96 // State Machine to send commands and data to LCD DISPLAY
97
98 always @(posedge CLK_400HZ or negedge iRST_N)
99     if (!iRST_N)
100     begin
101         state <= RESET1;
102     end
103     else
104     case (state)
105     RESET1:
106 // Set Function to 8-bit transfer and 2 line display with 5x8 Font size
107 // see Hitachi HD44780 family data sheet for LCD command and timing details
108     begin
109         LCD_E <= 1'b1;
110         LCD_RS <= 1'b0;
111         LCD_RW_INT <= 1'b0;
112         DATA_BUS_VALUE <= 8'h38;
113         state <= DROP_LCD_E;
114         next_command <= RESET2;
115         CHAR_COUNT <= 5'b00000;
116     end
117     RESET2:
118     begin
119         LCD_E <= 1'b1;
120         LCD_RS <= 1'b0;
121         LCD_RW_INT <= 1'b0;
122         DATA_BUS_VALUE <= 8'h38;
123         state <= DROP_LCD_E;
124         next_command <= RESET3;
125     end
126     RESET3:
127     begin
128         LCD_E <= 1'b1;
129         LCD_RS <= 1'b0;
130         LCD_RW_INT <= 1'b0;
131         DATA_BUS_VALUE <= 8'h38;
132         state <= DROP_LCD_E;
133         next_command <= FUNC_SET;
134     end
135 // EXTRA STATES ABOVE ARE NEEDED FOR RELIABLE PUSHBUTTON RESET OF LCD
136
137     FUNC_SET:
138     begin
139         LCD_E <= 1'b1;
140         LCD_RS <= 1'b0;
141         LCD_RW_INT <= 1'b0;
142         DATA_BUS_VALUE <= 8'h38;
143         state <= DROP_LCD_E;
144         next_command <= DISPLAY_OFF;
145     end
146
147 // Turn off Display and Turn off cursor
148     DISPLAY_OFF:
149     begin
150         LCD_E <= 1'b1;
151         LCD_RS <= 1'b0;
152         LCD_RW_INT <= 1'b0;
153         DATA_BUS_VALUE <= 8'h08;
154         state <= DROP_LCD_E;

```



```

155     next_command <= DISPLAY_CLEAR;
156 end
157
158 // Clear Display and Turn off cursor
159 DISPLAY_CLEAR:
160 begin
161     LCD_E <= 1'b1;
162     LCD_RS <= 1'b0;
163     LCD_RW_INT <= 1'b0;
164     DATA_BUS_VALUE <= 8'h01;
165     state <= DROP_LCD_E;
166     next_command <= DISPLAY_ON;
167 end
168
169 // Turn on Display and Turn off cursor
170 DISPLAY_ON:
171 begin
172     LCD_E <= 1'b1;
173     LCD_RS <= 1'b0;
174     LCD_RW_INT <= 1'b0;
175     DATA_BUS_VALUE <= 8'h0C;
176     state <= DROP_LCD_E;
177     next_command <= MODE_SET;
178 end
179
180 // Set write mode to auto increment address and move cursor to the right
181 MODE_SET:
182 begin
183     LCD_E <= 1'b1;
184     LCD_RS <= 1'b0;
185     LCD_RW_INT <= 1'b0;
186     DATA_BUS_VALUE <= 8'h06;
187     state <= DROP_LCD_E;
188     next_command <= Print_String;
189 end
190
191 // Write ASCII hex character in first LCD character location
192 Print_String:
193 begin
194     state <= DROP_LCD_E;
195     LCD_E <= 1'b1;
196     LCD_RS <= 1'b1;
197     LCD_RW_INT <= 1'b0;
198     // ASCII character to output
199     if (Next_Char[7:4] != 4'h0)
200         DATA_BUS_VALUE <= Next_Char;
201         // Convert 4-bit value to an ASCII hex digit
202     else if (Next_Char[3:0] >9)
203         // ASCII A...F
204         DATA_BUS_VALUE <= {4'h4, Next_Char[3:0]-4'h9};
205     else
206         // ASCII 0...9
207         DATA_BUS_VALUE <= {4'h3, Next_Char[3:0]};
208     // Loop to send out 32 characters to LCD Display (16 by 2 lines)
209     if ((CHAR_COUNT < 31) && (Next_Char != 8'hFE))
210         CHAR_COUNT <= CHAR_COUNT + 1'b1;
211     else
212         CHAR_COUNT <= 5'b000000;
213     // Jump to second line?
214     if (CHAR_COUNT == 15)

```

```

215     next_command <= LINE2;
216 // Return to first line?
217     else if ((CHAR_COUNT == 31) || (Next_Char == 8'hFE))
218         next_command <= RETURN_HOME;
219     else
220         next_command <= Print_String;
221 end
222
223 // Set write address to line 2 character 1
224 LINE2:
225 begin
226     LCD_E <= 1'b1;
227     LCD_RS <= 1'b0;
228     LCD_RW_INT <= 1'b0;
229     DATA_BUS_VALUE <= 8'hC0;
230     state <= DROP_LCD_E;
231     next_command <= Print_String;
232 end
233
234 // Return write address to first character position on line 1
235 RETURN_HOME:
236 begin
237     LCD_E <= 1'b1;
238     LCD_RS <= 1'b0;
239     LCD_RW_INT <= 1'b0;
240     DATA_BUS_VALUE <= 8'h80;
241     state <= DROP_LCD_E;
242     next_command <= Print_String;
243 end
244
245 // The next three states occur at the end of each command or data transfer to the LCD
246 // Drop LCD E line - falling edge loads inst/data to LCD controller
247 DROP_LCD_E:
248 begin
249     LCD_E <= 1'b0;
250     state <= HOLD;
251 end
252 // Hold LCD inst/data valid after falling edge of E line
253 HOLD:
254 begin
255     state <= next_command;
256 end
257 endcase
258 endmodule
259
260 module LCD_display_string(index,out,hex0,hex1);
261 input [4:0] index;
262 input [3:0] hex0,hex1;
263 output [7:0] out;
264 reg [7:0] out;
265 // ASCII hex values for LCD Display
266 // Enter Live Hex Data Values from hardware here
267 // LCD DISPLAYS THE FOLLOWING:
268 //-----
269 //| Count=XX          |
270 //| DE2              |
271 //-----
272 // Line 1
273 always
274     case (index)

```

```
275     5'h00: out <= 8'h43;
276     5'h01: out <= 8'h6F;
277     5'h02: out <= 8'h75;
278     5'h03: out <= 8'h6E;
279     5'h04: out <= 8'h74;
280     5'h05: out <= 8'h3D;
281     5'h06: out <= {4'h0,hex1};
282     5'h07: out <= {4'h0,hex0};
283 // Line 2
284     5'h10: out <= 8'h44;
285     5'h11: out <= 8'h45;
286     5'h12: out <= 8'h32;
287     default: out <= 8'h20;
288     endcase
289 endmodule
```

A.3 reset_delay.v

```
1
2 module    Reset_Delay(iCLK,oRESET);
3 input     iCLK;
4 output reg oRESET;
5 reg       [19:0] Cont;
6
7 always@(posedge iCLK)
8 begin
9     if(Cont!=20'hFFFFFF)
10        begin
11            Cont    <=    Cont+1'b1;
12            oRESET   <=    1'b0;
13        end
14    else
15        oRESET    <=    1'b1;
16 end
17
18 endmodule
```


ANEXO B – Debounce

O código do Debounce foi desenvolvido por Tony Storey e é apresentado abaixo sem as alterações realizadas.

```

1 // DeBounce_v.v
2
3
4 ////////////////////////////////////////////////// Button Debouncer //////////////////////////////////////////
5 //*****
6 // FileName: DeBounce_v.v
7 // FPGA: MachX02 7000HE
8 // IDE: Diamond 2.0.1
9 //
10 // HDL IS PROVIDED "AS IS." DIGI-KEY EXPRESSLY DISCLAIMS ANY
11 // WARRANTY OF ANY KIND, WHETHER EXPRESS OR IMPLIED, INCLUDING BUT NOT
12 // LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A
13 // PARTICULAR PURPOSE, OR NON-INFRINGEMENT. IN NO EVENT SHALL DIGI-KEY
14 // BE LIABLE FOR ANY INCIDENTAL, SPECIAL, INDIRECT OR CONSEQUENTIAL
15 // DAMAGES, LOST PROFITS OR LOST DATA, HARM TO YOUR EQUIPMENT, COST OF
16 // PROCUREMENT OF SUBSTITUTE GOODS, TECHNOLOGY OR SERVICES, ANY CLAIMS
17 // BY THIRD PARTIES (INCLUDING BUT NOT LIMITED TO ANY DEFENSE THEREOF),
18 // ANY CLAIMS FOR INDEMNITY OR CONTRIBUTION, OR OTHER SIMILAR COSTS.
19 // DIGI-KEY ALSO DISCLAIMS ANY LIABILITY FOR PATENT OR COPYRIGHT
20 // INFRINGEMENT.
21 //
22 // Version History
23 // Version 1.0 04/11/2013 Tony Storey
24 // Initial Public Release
25 // Small Footprint Button Debouncer
26
27 `timescale 1 ns / 100 ps
28 module DeBounce
29 (
30     input                clk, n_reset, button_in,
31         // inputs
32     output reg           DB_out
33         // output
34 );
35
36 //// ----- internal constants -----
37     parameter N = 11 ; // (2N - 1) / 38 MHz = 32 ms debounce time
38
39 ////----- internal variables -----
40     reg [N-1 : 0] q_reg; // timing
41         regs
42     reg [N-1 : 0] q_next;
43     reg DFF1, DFF2;
44         // input flip-flops
45     wire q_add;
46         // control flags
47     wire q_reset;
48
49 //// -----
50
51 ////contentious assignment for counter control
52     assign q_reset = (DFF1 ^ DFF2); // xor input flip flops to look
53         for level chage to reset counter

```

```

45     assign q_add = ~(q_reg[N-1]);           // add to counter when q_reg msb
        is equal to 0
46
47     /// combo counter to manage q_next
48     always @ ( q_reset, q_add, q_reg)
49     begin
50         case( {q_reset , q_add})
51             2'b00 :
52                 q_next <= q_reg;
53             2'b01 :
54                 q_next <= q_reg + 1;
55             default :
56                 q_next <= { N {1'b0} };
57         endcase
58     end
59
60     /// Flip flop inputs and q_reg update
61     always @ ( posedge clk )
62     begin
63         if(n_reset == 1'b0)
64             begin
65                 DFF1 <= 1'b0;
66                 DFF2 <= 1'b0;
67                 q_reg <= { N {1'b0} };
68             end
69         else
70             begin
71                 DFF1 <= button_in;
72                 DFF2 <= DFF1;
73                 q_reg <= q_next;
74             end
75         end
76
77     /// counter control
78     always @ ( posedge clk )
79     begin
80         if(q_reg[N-1] == 1'b1)
81             DB_out <= DFF2;
82         else
83             DB_out <= DB_out;
84         end
85
86     endmodule

```